華中科技大學

《软件工程》项目报告

题目：基于识谱演奏一体化的音乐教学播放应用

课程名称：软件工程

专业班级：**CS2011**

组　　名：乐缘小队

同组成员：学号：**U202013194**

姓名：张子豪

学号：**U202010755**

姓名：路昊东

学号：**U202011369**

姓名：薛江博

学号：**U202010889**

姓名：吴潇

指导教师：冯琪

报告日期：**2022.12.13**

计算机科学与技术学院

任 务 书

一 总体要求

1. 综合运用软件工程的思想，协同完成一个软件项目的开发，掌软件工程相关的技术和方法；
2. 组成小组进行选题，通过调研完成项目的需求分析，并详细说明小组成员的分工、项目的时间管理等方面。
3. 根据需求分析进行总体设计、详细设计、编码与测试等。

二 基本内容

根据给出的题目任选一题，自行组队，设计与开发中软件过程必须包括：

1. **问题概述、需求分析：**正确使用相关工具和方法说明所开发软件的问题定义和需求分析，比如 NABCD 模型，Microsoft Visio，StarUML 等工具 (20%)；
2. **原型系统设计、概要设计、详细设计：**主要说明所开发软件的架构、数据结构及主要算法设计，比如墨刀等工具（35%）；
3. **编码与测试：**编码规范，运用码云等平台进行版本管理，设计测试计划和测试用例（30%）；
4. **功能创新：**与众不同、特别吸引用户的创新（10%）；
5. **用户反馈：**包括用户的使用记录，照片，视频等（5%）。

目 录

任务书 1
1 问题定义 3
1.1 项目背景及意义 3
1.2 项目基本目标 4
1.3 可行性分析 6
1.4 人员管理和项目进度管理 7
2 需求分析 10
2.1 需求分析概述 10
2.2 UML 相关需求分析图 11
2.3 原型系统设计 14
3 概要设计和详细设计 19
3.1 系统结构 19
3.2 对象模型和关键数据结构设计 20
3.3 关键算法设计 29
3.4 数据管理说明 33
4 实现与测试 34
4.1 实现环境和代码管理 34
4.2 关键函数说明 36
4.3 测试计划和测试用例 38
4.4 结果分析 43
5 总结 44
5.1 用户反馈 44
5.2 总结 44
6 体会 47

附录 53

1 问题定义

1.1 项目背景与意义

1.1.1 项目背景

这里我们不妨从实际背景和我们的自我愿景来谈这个作品。

关于实际背景，我们不得不谈到的，是 MIDI 文件格式。

对于现实中的人，我们演奏音乐的步骤可以分为以下部分：识谱，以及把识别出来的记号转换到乐器演奏上，比如说看到五线谱里的音符，于是我们就知道按什么键，弹什么弦，唱什么音。

我们听到声音的格式或者称之为“样式”，几乎完全取决于使用的乐器的种类和样式；然而在数字音乐世界里，充当乐器的是数码设备，多数时候是电脑。但无论是什么设备，大家都是把物理的音乐信号数字化来处理的，设备是无法对五线谱文件进行直接解析的，它们需要一种新的记录音乐的方法，这就是 MIDI。

那数字设备怎么演奏音乐的呢？首先由识谱者，比如我们，帮电子设备识谱，然后把曲谱转化为 MIDI 文件（转换的方式有很多，可以完全用电脑的键盘和鼠标，也可以用看起来很像钢琴或电子琴的数字音乐键盘）；之后我们会把 MIDI 传给数字设备，数字设备使用算法将 MIDI 信号与我们可识别的乐谱对应起来；最后就是数字设备“演奏”这些乐谱。实际上，数字设备就是用 MIDI 来演奏的，只不过我们在屏幕上可以看到类现实世界的视觉反馈。与之前描述相对应的，我们听到声音的格式或者称之为“样式”，取决于数字设备当时加载的音色。

综上，如果把乐谱看作是人类音乐世界里一种用来相互交流的语言和文字，那么 MIDI 就是电子设备的语言和文字，一种通用的、另类的乐谱。

那我们的愿景是什么？

音乐的美在于感染力在于共享，而开源同样是程序员的浪漫，我们因为对音乐和编程的热爱而结缘，我们设计开发的应用同样如此，其目的就是为了让更多人发现自己身边的优美音乐，尽情享受并能自己去亲身演奏、去创作。

本应用，omg-score，就是基于 MIDI 文件的特殊特性和我们的愿景开发的一个小而美的开源 web 应用。

我们会给用户提供优质的曲谱 mp3 文件转谱，以及动态实时的乐器演奏等服务，让用户能够根据软件上的学习内容，快速入门掌握基本知识，同时提高

用户的学习兴趣，并完全可以帮助用户掌握乐谱实际上的弹奏，只需要一个 web 界面，就可以帮助我们得到喜欢的曲谱，自己弹奏出美妙的乐曲。

1.1.2 项目特色与意义

本 omg-score 应用是一个小而美的开源 web 应用。

用户无需提供任何形式的个人信息，唯一需要共享（上传）的，就是自己拥有的 mp3 音乐文件；

相似的，用户使用无需任何形式的系统和环境限制，只要有网络，只需要一个 web 界面，就可以体验本应用的全部功能。

我们要做的，就是给用户提供优质的曲谱 mp3 文件转谱为 MIDI 文件，以及动态的、实时的乐器演奏等服务，让用户能够根据软件上的学习内容，快速入门掌握基本知识，同时提高用户的学习兴趣，并完全可以帮助用户掌握乐谱实际上的弹奏，并能自行保存转换好的 MIDI 文件，进行不同形式的学习和二次创作等等。由于时间限制，目前仅以钢琴这一种乐器为 demo，后期会持续更新。

经调研得到，我们的想法和形式在国内外免费和开源软件领域都算是独一无二，即使各大付费软件也只是涉及转谱功能。

识谱是音乐学习的基础，演奏是最终目的之一。本次设计可以做到对音乐进行较为准确的自动识谱，并能提供有趣的自动教学。想必对类似应用比如各大听歌软件、播放器等会有一定的启发式的借鉴意义；而且本应用可以直接用于音乐检索、音乐乐谱分析等热点领域，是一个非常有前景的研究领域；同时，本应用对于音乐爱好者和初级学习者提供了一个免费、便利、快捷的良好平台，也属于一个富有延展性和包容性的应用，它的开发和优化将永无止境，也有着良多的意义。

1.2 项目基本目标

本次项目的基本目标是开发一款小而美的 web 音乐转谱播放应用。针对这一目标，我们小组研究了多款相关 web 应用、移动端和 PC 端的 app 或者小程序，分析出了我们需要实现的基本功能和基础架构。同时，我们研究了类似竞品的不足之处，以此来在我们的项目开发中避免这些问题，让用户获得更好的体验，帮助广大音乐专业人士和音乐爱好者实现将任意一段音频文件转化为

MIDI 文件，并在构建的虚拟乐器上实时显示出来，同时完成分析和记录乐谱的任务功能。

最后，我们将研究成果绘制成如图 1-1 所示的思维导图。

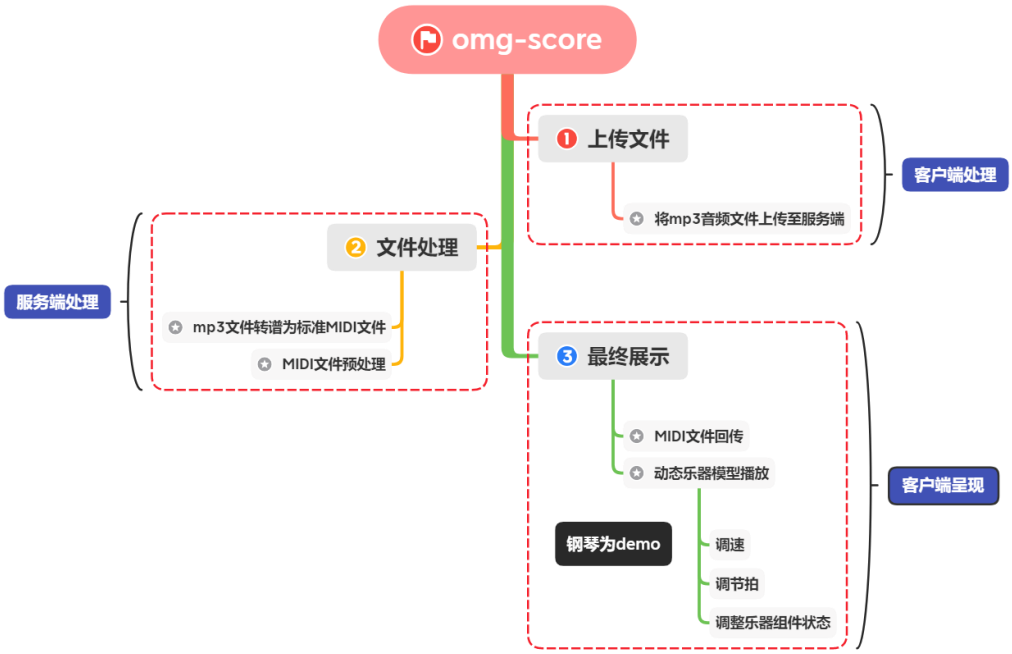


图 1-1 omg-score 思维导图

根据如上的思维导图，我们可以基本确定项目目标的系统设计，将项目基本目标划分为下述若干个小目标：

- 需要实现上传文件的功能： 不同形式的音频文件（比如.mp3 格式）在浏览器网页界面可以直接上传文件到服务端，并在服务端实现存储功能；
- 需要实现文件处理的功能： 上传后的音频文件（比如.mp3 文件）在服务端需要被转为标准的 MIDI 文件，同时要将 MIDI 文件转储在服务端；之后需要对 MIDI 文件进行一定形式的处理，转化成易于返回的格式，最后需要将 MIDI 文件内容和基本信息回传给浏览器客户端；
- 需要实现最终在 web 界面上的展示功能： 客户端界面上将出现一个 3D 可调节方向、大小、位置，乃至按键/弦的颜色的一种或多种乐器模型（这里暂时以钢琴为 Demo），以及基本的调节按键比如速率调节、节拍调节、上下曲切换、播放、暂停、停止等。当客户端接收到了服务端回传的 MIDI 文件后，会迅速将其转化为可供浏览器播放的形式，并选择性下载到客户端本地的磁盘空间内，当用户点击“播放”按键，浏览器会将处理好的 MIDI 文件播放，并显示到乐器的按键跳动、变色等变化中，同时用户可根据界面上的组件进行调控等；
- 去除/避免任何形式的注册登录、认证等个人信息收集环节，不但利于网

战备案同时维护我们的设计初衷；

- **延展目标：**
  - 丰富展示的乐器形式，甚至实现在同一个界面上显示多种乐器，以便对大型复杂弦乐等进行解析；
  - 优化服务端处理速度以及客户端相应速度；
  - 提高网站安全性；
  - 增加多种音频文件的处理比如.wav、.flac、.ape、.aac 等文件格式；
  - 增加匿名评论功能，乃至匿名论坛，在确保文明交流的前提下分享自己的二次创作以及对于乐曲的评价；

延展目标正反映出我们的应用具有极强的包容性和扩展性，我们将继续开发，继续努力！

1.3 可行性分析

1.3.1 技术可行性

通过查询各种资源、资料，可以看到目前在市面上已经存在各种各样的听音识谱的手机软件或者电脑软件，此外也了解到听音识谱的基本步骤和具体实现流程，说明开发此项目是**存在可行方法**的，所以本听音识谱项目是具有一定的技术可行性。在具体实现上：

- 后端实现 mp3 文件转化为 MIDI 文件输出使用 JAVA 语言进行开发设计
- 前端开发中使用 Html、Css、Javascript 实现

由于小组成员对于相关的语言和程序设计均有一定的基础，加之站在前人的肩膀上，在一定的有限的且尽可能短的时间内，凭借最小的代价，是可以解决其中可能会遇到的一些技术问题，并最终把本项目开发实现出来。

- **投入方面：**
  - 从**开发成本**上对项目目标进行分析，本次项目完成所需成本不算太高。
  - 在**时间成本**上，小组成员可以完成其他课业要求的基础上，利用课堂课余时间完成项目。
  - 在**经济成本**上，本次项目不需要专门的开发设备或是开发地点，但需要一定成本来申请购买服务器，但总体成本预估在可接受范围之内。同时后端实现上使用 JAVA 语言，前端开发中使用 Html、Css、Javascript 实现，不会产生过多的额外经济成本。

- **效益方面：**前期是网上免费的提供别人使用，因此并没有实际的利益，但是可以传播影响，在后期被其他公司收购时候能够创造出价值。

1.3.2 经济可行性

总体经济可行性分析是：系统开发耗资少，且符合了当今社会中方便和实用的想法，因此该软件研发成果，带来的效益必定大于投入的效益。

- **收益/投资比方面：**投资的花费并不大，但是收益是不确定的，更多的是一种隐性收益。
- **投资回收周期方面：**投资低，因而风险小，从此考虑到收益大概需要时间的沉淀，不能给准确的计算。

1.3.3 社会可行性

- 从**市场需求**方面，对项目目标进行分析，我们认为当前听音识谱应用有着较为广阔的市场应用前景。
- 从**用户需求**上来看，目前有许多的音乐爱好者在听到一段音乐时试图把其乐谱分析出来仔细研究，但却介于目前市场上并没有太多的实用免费的软件供其使用，而这些用户正需要我们所开发的这一项产品。
- 从**市场竞争**上来看，虽然传统的听音识谱应用程序仍占有大量的用户市场，且在 Web 应用端的市场上，仍会存在较多的目标用户，因此我们认为我们的产品可以在市场上站住脚跟。
- 在**产品内容**上，我们会对其他的听音识谱应用取其精华、去其糟粕，以争夺目标用户。
- 在**产品推广**上，我们可以借助社交平台以及校内校外宣传打下用户基础，再进一步地向更广大的用户群体迈进。

1.4 人员管理和项目进度管理

在确立了项目基本目标，完成了项目可行性分析后，本小组便开始制定项目进度计划，并根据计划内预计的任务负担合理地为每一位小组成员分配各个阶段的任务，并制定完成期限。

1.4.1 人员分工管理

本小组共计有 4 名成员。鉴于本次项目参与人员较少，但工程量比较大，我们决定采用民主制组织模式进行交流沟通，各自可以根据自身的能力以及对项目的期望，自由地选择想要负责的任务，最终的小组具体分工如下（排名不分先后，按姓名首字母排序）：

- 路昊东：选题调研、系统设计与前后端框架搭建、前端 3d 模型、MIDI 演奏逻辑等功能实现、后端（服务端）功能实现、前后端交互功能实现；
- 吴潇：选题调研、原型设计、前端界面设计与美化、前端通信传参部分、代码测试；
- 薛江博：选题调研、后端 MP3 文件到 MIDI 文件的转换、MIDI 文件存储记录、代码测试；
- 张子豪：选题调研、NABCD 模型设计、后端将 MP3 文件转化为 MIDI 文件代码部分、代码测试、软件过程模型设计、UML 相关需求分析图设计。

1.4.2 项目进度管理

在进行初步的需求分析后，我们得到如图 1-2 所示的初步的系统结构设计图。该设计图将各个需要实现的产品功能划分成 3 个大模块（前端、后端、前后端连接）与若干个子模块，根据各个功能模块预估的工作量，我们可以完成初步的进度计划与分工。



图 1-2 omg-score 系统结构图

下表 1-1 是根据上述系统结构图所分的功能模块以及项目开发过程的其他

任务，得到的预计的项目完成时间的计划表。考虑到项目进行期间的课业压力以及其他的实验任务，我们对每一项任务的计划时间进行了延长。

- 设计阶段：分为原型设计、需求规格说明等，由所有组员共同完成。
- 开发阶段：分为三部分的功能模块设计，由队长负责指导、协助，其他三位成员各完成一部分。
- 测试阶段：分为系统测试和代码测试。
- 发布阶段：分为产品发布后的用户反馈收集与处理。

表 1-1 项目进度计划表

任务规划	开始时间	计划天数/天	具体说明
需求分析	9.30	3	绘制需求分析模型（UML）
系统原型设计	10.7	6	利用墨刀完成可互动的系统原型
前端基础图形化模块设计	10.14	3	设计基础界面，准备好位置分配给后续不同的设计模块
3d 钢琴模块设计	10.20	5	实现 3d 立体的钢琴设计，包括按下按键、旋转伸缩等功能
mp3 到 midi 文件的转换模块设计	10.20	10	实现对输入的 midi 文件进行解析并转换为便于记录的 midi 文件的功能
midi 文件存储记录模块设计	10.23	5	实现对于转换完成的 midi 文件进行存储记录，以待后面导入到 3d 钢琴中进行播放的功能
前后端跨域模块功能设计	10.25	5	实现前后端在服务器和不同 IP 的客户端进行通信的功能
web 页面中控件各功能模块设计	10.25	8	实现控件设计，包括对于控件中各个功能如调节音高、改变按键颜色、选择 midi 文件进行播放、暂停功能实现
前后端连接输入 mp3 接口模块设计	11.4	6	实现将任意一段 mp3 文件输入后，可在后端自动转换为 midi 文件的功能
mp3 文件导入 web 页面的 3d 钢琴模块设计	11.4	6	实现将转换完成的 midi 文件导入到 web 页面中的 3d 钢琴进行实时播放的功能

2 需求分析

2.1 需求分析概述

2.1.1 N (Need 需求）：

很多音乐爱好者想把喜欢的歌曲可视化。通过人耳识谱需要很高的音乐水平，又耗时很多，容易出错。现有的听音识谱软件往往不专注于钢琴谱，且又收费。因而开发一款专注于钢琴的识谱演奏应用具有很大的市场。

2.1.2 A (Approach 方法)：

先将 mp3 文件转为 MIDI 文件，再将 MIDI 文件可视化，自动在 web 界面的钢琴键上弹奏。

mp3 转 MIDI 使用机器学习实现，MIDI 转为在钢琴键上弹奏通过 web 程序实现。

2.1.3 B (Benefit 好处)：

- 开发的是 web 应用而非手机 apk 应用或电脑 exe 文件，从而没有系统和环境的限制，普适性强。
- 相比于传统的听音识谱软件，我们的应用可以导出 MIDI 文件，并且可以显示乐谱和直观地在模拟钢琴上弹奏出来。
- 可以通过按键盘来弹奏钢琴，实际上就是开发了一架虚拟钢琴，从而可以根据谱子弹奏，而这是传统识谱软件所不具有的功能。

2.1.4 C (Competitors 竞争)：

我们的竞争对手主要是传统的钢琴演奏和制谱软件，如浮浮雷达、爱听、歌姬歌姬和酷我畅听。这些应用往往需要开会员才能够提供优质服务，并且并不是专注于钢琴谱的听音扒谱，这使得钢琴爱好者的需求不能被很好地满足。

除了扒谱应用以外，还有一批靠人耳听音识谱的音乐爱好者，他们往往水平很高，因而群体数量少，普通音乐爱好者无法达到这种水平，所以对我们的产品影响很小。

对比后可知，我们的应用既提供了针对钢琴谱的专业服务，又不收费，还满足了普通音乐爱好者的需求。

2.1.5 D (Delivery 推广)：

- 借助知名的社交平台进行营销推广，如微信公众号运营，朋友圈推广等；
- 通过线上线下社交活动推广，例如在学院年级群、线下路演等场景中让同学们尝试并帮助推广；
- 借助微信小程序平台推广，让听音识谱 web 应用推送至各个用户手中体验；
- 向正在使用小程序的用户征集反馈意见，并发送一定的福利，产生扩大效应，吸引更多用户加入。

2.2 UML 相关需求分析图

2.2.1 系统用例图

我们使用用例图对用户的需求和操作做出合理分析，进一步明确目标，如图 2-1 所示。

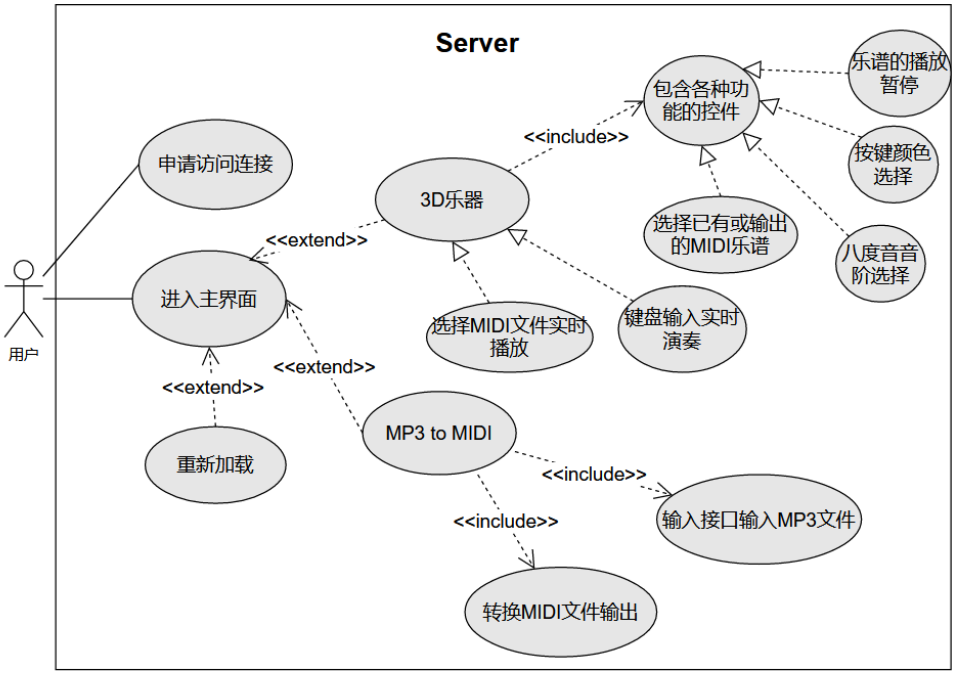


图 2-1 3D 乐器 WEB 应用系统用例图

由上述的非功能性需求和需求优先级的分析，清晰用户的分级操作。

用户进入我们的 web 界面后，等待片刻便可看到我们的 3D 乐器界面和用户控制界面。这时候用户可以选择在我们预设好的 MIDI 文件夹内选择一首自己喜欢的音乐，或者直接在本地选择一个 MIDI 格式的文件。

此外，我们还提供了一个将 MP3 格式的文件转成 MIDI 格式文件的功能，用户可以在自己的本地路径下选择一个 mp3 文件并导入，我们设计的软件可以将其转化成 MIDI 文件。接下来，用户可以在界面上看到歌曲的实时演奏，还可以对其进行控制。

2.2.2 E-R 图

根据上述系统用例图分析，我们将对其进行具体的实体-联系分析，从而获取所需的实体与属性，以及实体间的联系关系。记账系统 E-R 图如图 2-2 所示。

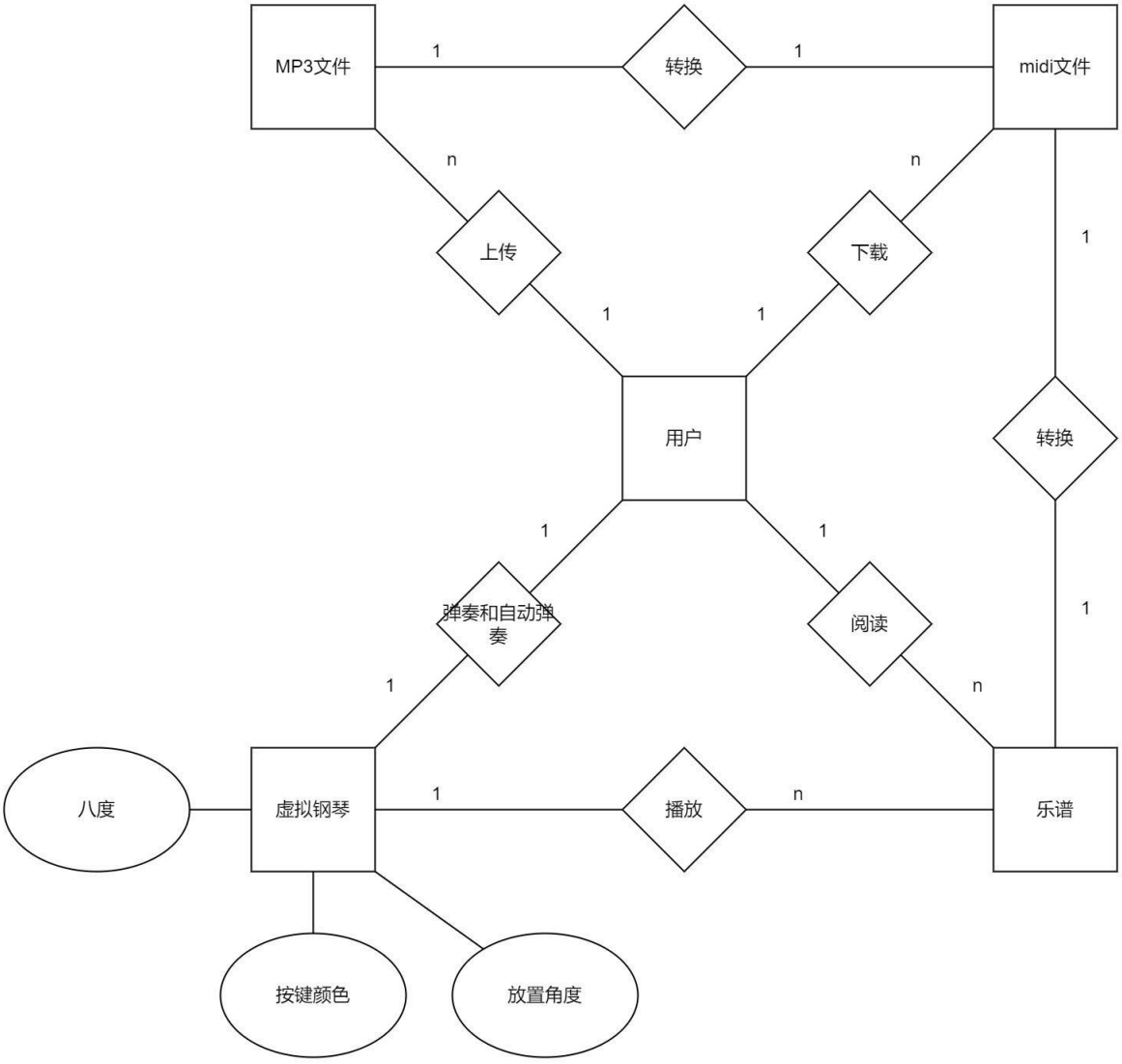


图 2-2 3D 乐器 WEB 应用 E-R 图

主体有 MP3 文件、MIDI 文件、用户、乐谱、虚拟钢琴等。一个 MP3 文件可以对应一个 MIDI 文件，一个用户可以上传多个 MP3 文件，并下载多个 MIDI 文件。一个 MIDI 文件可以转化成一个乐谱，供用户阅读。一个用户可以控制一个虚拟钢琴，一个虚拟钢琴可以播放多个乐谱。

2.2.3 数据流图

数据在系统内的逻辑流向和变化主要体现在 MP3 到 MIDI 文件的转换阶段和 3D 乐器弹奏操作阶段。

绘制如图 2-3 所示的数据流图，分析系统内的数据流向。

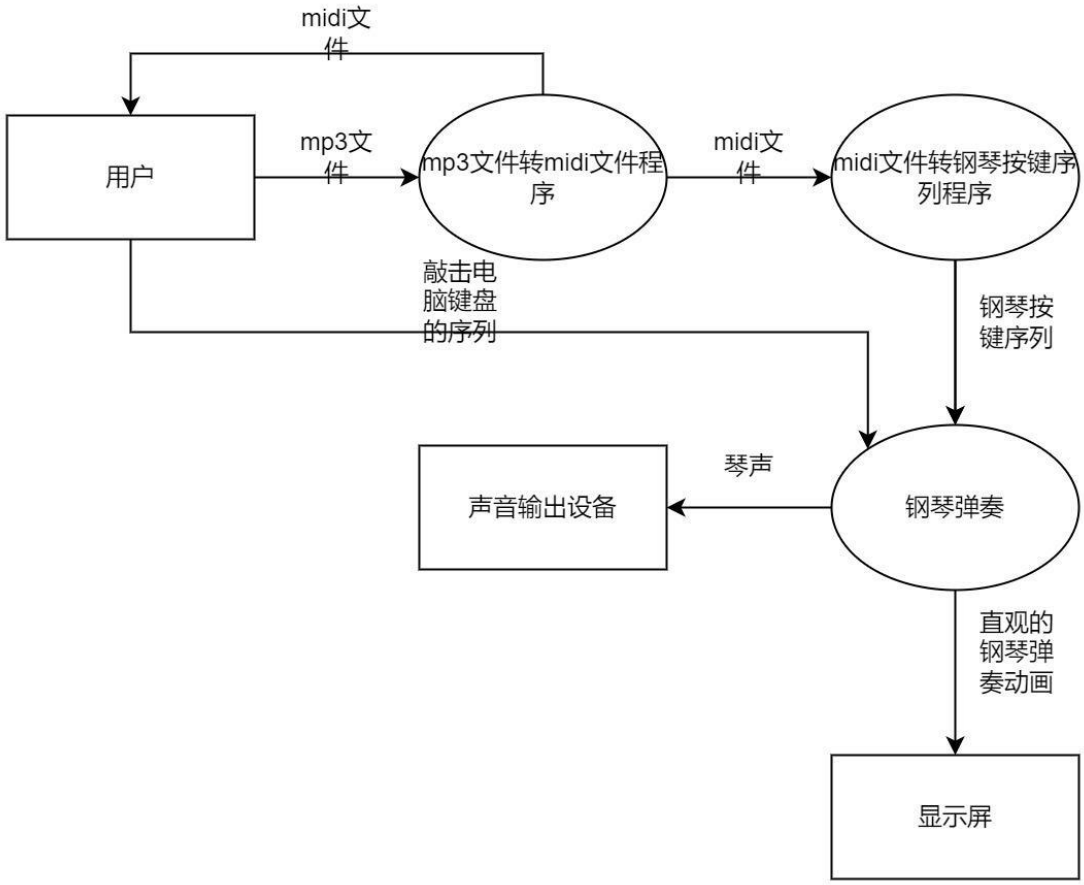


图 2-3 3D 乐器 WEB 应用数据流图

用户输入 MP3 文件后，可以将其转换成 MIDI 文件，并提供给用户和后端。虚拟钢琴可以通过用户控制，也可以自己对指定歌曲进行演奏。钢琴演奏出的声音讯号通过电脑的默认输出设备进行播放，并直观地显示在显示屏上。

这其中参与到的数据类型有：声音信息、MP3 文件信息、MIDI 文件信息、I/O 信息。

其中 I/O 信息包括：键盘输入、文件输入、文件输出、声音输出、显示输出。

2.2.4 用例交互图

根据相关用例分析，可以清晰的展示各功能间的层级关系。通过用户的一次 3D 乐器 WEB 网页应用的用例，绘制如图 2-4 所示的用例交互图。

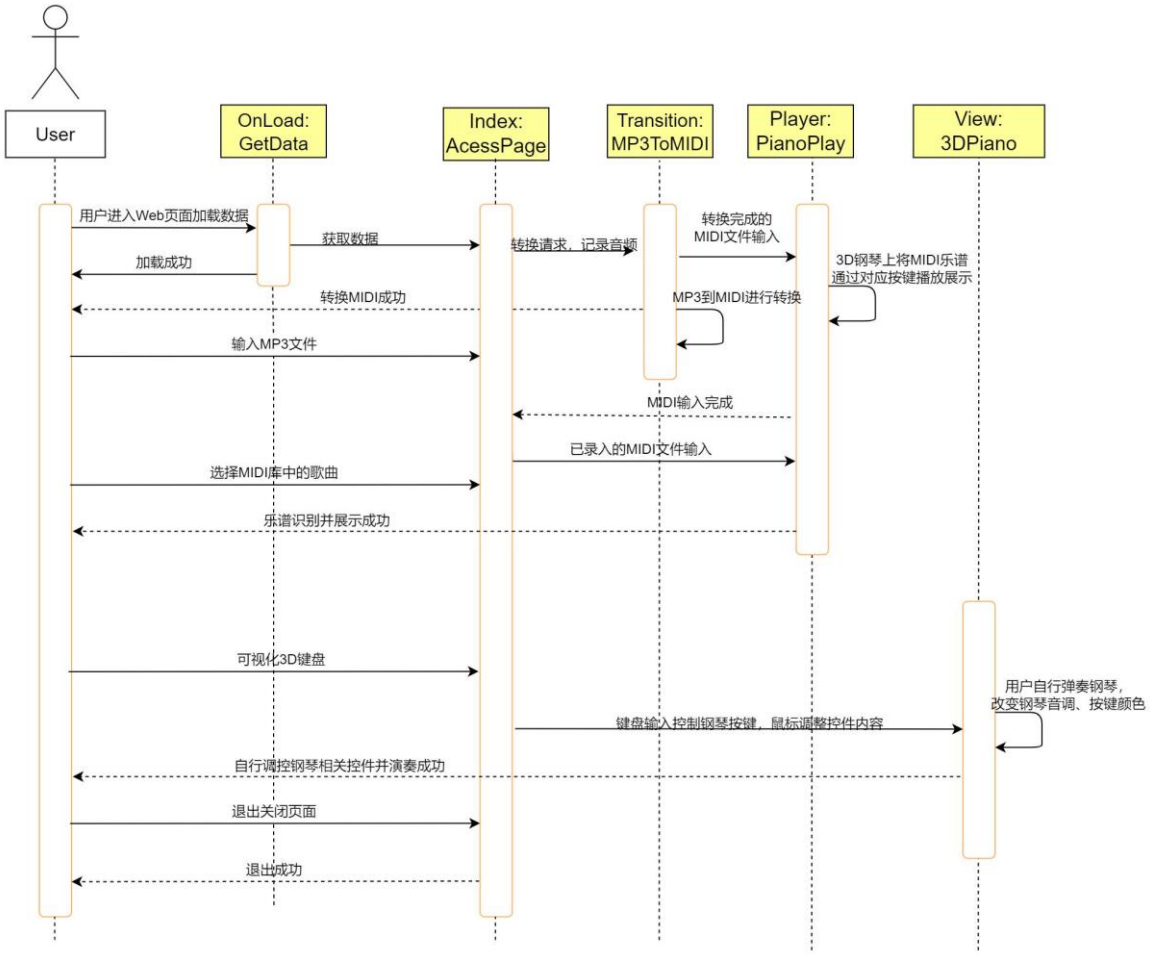


图 2-4 3D 乐器 WEB 应用用例交互图

对上图进行如下详细说明：

- OnLoad：将输入的文件加载到软件
- Index：用户控制界面
- Transtion：MP3 转 MIDI
- Player：MIDI 播放器
- View：3D 虚拟钢琴界面

2.3 原型系统设计

运用工具（例如墨刀）设计原型系统，从而更准确说明主要功能和用户交互界面。

原型文件 URL：https://orgnext.modao.cc/app/unhq0y9Zrkg9ukRg7qvSJB#原型演示文稿-分享

2.3.1 主要功能

❖ **用户可以在 MIDI 文件库里寻找自己喜欢的歌曲。**

我们为用户一共提供了 21 首著名钢琴曲的 MIDI 文件，用户可以对其进行自由选取，并进入接下来的步骤。

同时，用户也可以输入 MP3 类型的文件，我们可以进行读取，并将其播放出来。

以下是我们提供的 MIDI 文件，它们都是世界上著名的钢琴曲：

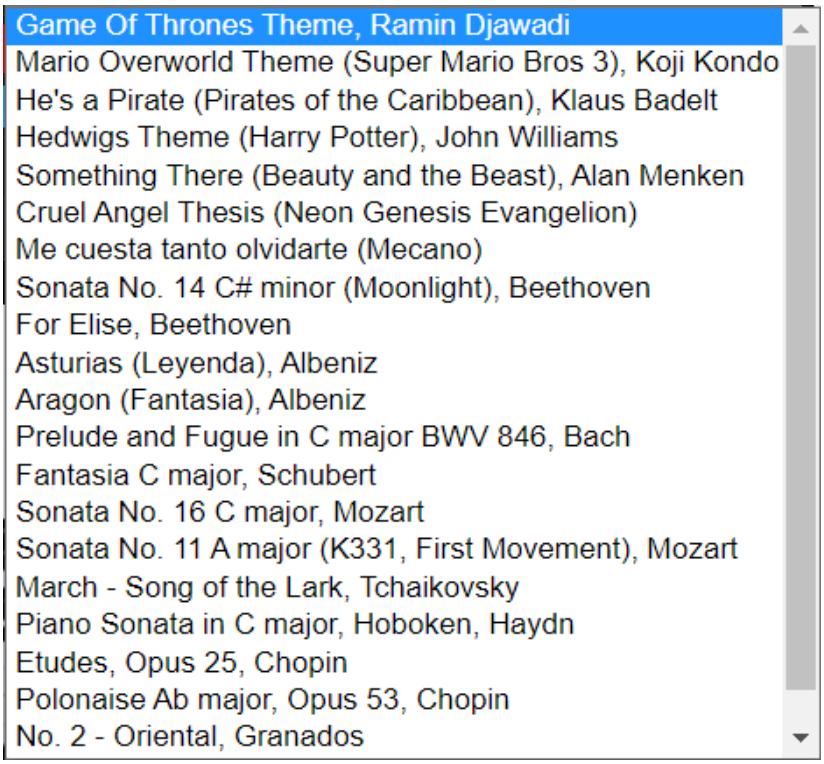


图 2-5 提供的已经建好的 MIDI 库

❖ **将 MIDI 文件播放出来，并实时显示在钢琴界面上。**

按下控制界面的 *play* 按钮，web 就可以对选中的歌曲进行播放。

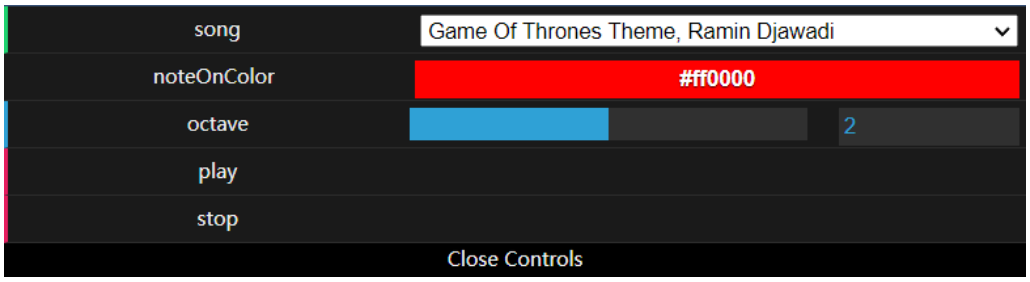


图 2-6 控制界面

我们可以通过下方的界面来观察琴键的变化。



图 2-7 MIDI 文件播放出来后，实时显示在钢琴上

其中控制界面的 noteOnColor 按钮控制着琴键按下去的颜色，默认为红色（#ff0000），也可以自己进行调整。

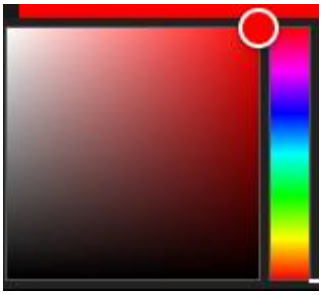


图 2-8 为用户提供的颜色选择窗口

❖ **通过电脑键盘控制钢琴键**

- 电脑键盘上的 *Q* 对应着钢琴上的中央 C，以此类推。每按下键盘上的某些键，界面就会显示相应的琴键，并且发出其对应的声音。
- 用户还可以同时按下许多键，钢琴可以同时演奏出它们的声音。

这样以来，用户就可以演奏出自己想要的内容。



图 2-9 通过电脑键盘进行控制

❖ **进行八度平移操作**

如图 3-2 所示，*octave* 按钮代表着按下电脑键盘后，琴键的位置，默认值为 2。

我们提供了 0~4，并且以 0.1 为分度值的选择。数值每加 1，琴键就往右移一个八度；数值每减 1，琴键就往左移一个八度。



图 2-10 octave 为 3 时，按下 Q 的界面

❖ **旋转、缩放**

我们将钢琴设计成一个可以围绕着一个中心进行旋转的模型。通过鼠标上的滚轮，可以对画面进行缩放。以便用户看到更多的细节。按住鼠标左键并移动，就可以对钢琴进行旋转。



图 2-11 旋转缩放后的界面

2.3.2 用户交互界面

展示给用户的 web 界面如图 2-12 所示：

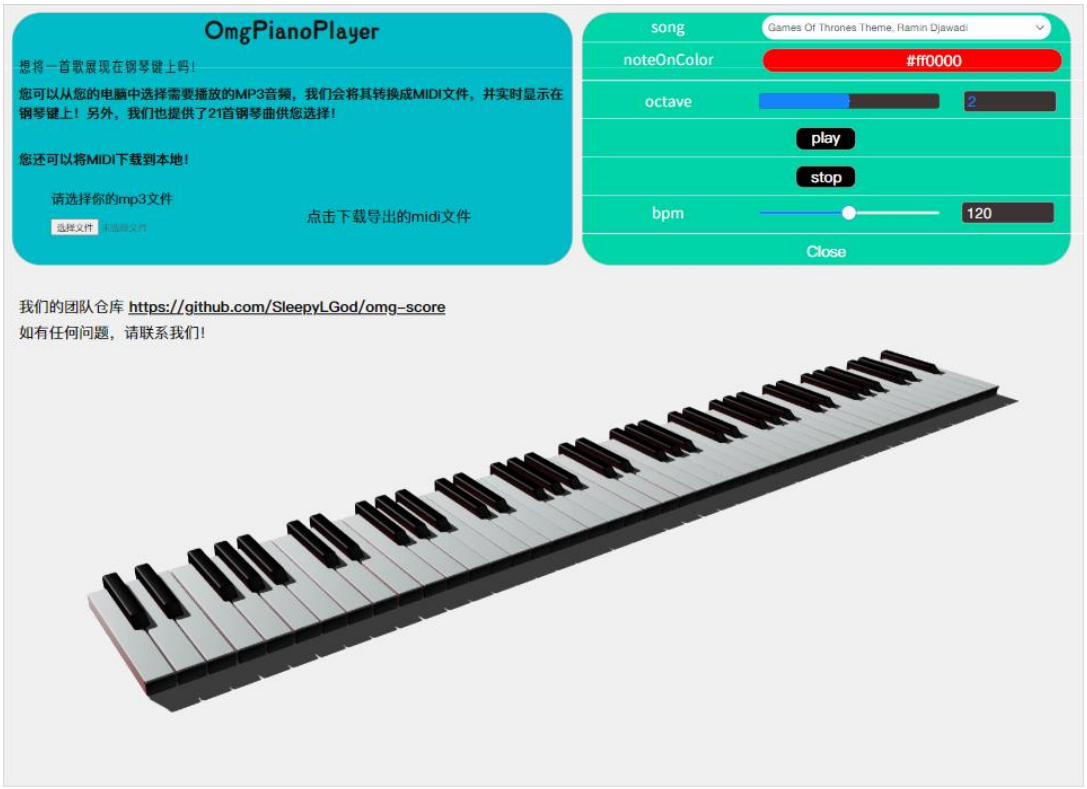


图 2-12 用户交互界面

这是用户一进入界面就会看到的交互面板：

- 左上角是我们的信息提示窗口。用户可以上传 MP3 文件，并点击下载相应的 MIDI 版本。
- 右上角是用户控制窗口，用户可以在里面对虚拟钢琴进行控制。
- 下面是软件的主体：虚拟钢琴，可进行自由旋转、位移和放大缩小。

3 概要设计和详细设计

3.1 系统结构

整个 **omg-score** 项目可以分解成多个模块，依靠模块自身功能的实现以及模块之间的协同合作达到预期的效果。对系统结构进行分析，可以将其分为两大模块：MIDI Generator 和 MIDI Player。

MIDI Generator 是类似服务端（后端）的模块，首先负责将音频文件比如.mp3 文件转成 midi 文件，客户端的用户选择本地自己喜欢的乐曲音频文件，通过网络传输给 Generator 服务端，Generator 负责将其转换成 midi 格式的文件，然后传给客户端，进行曲谱的展示。

MIDI Player 则是一个类似客户端（前端）的模块，首先负责将音频文件比如.mp3 格式的文件传输给服务端，其次负责接收服务端返回的 midi 文件，并将其再次解析成“动态”曲谱，在界面上显示的 3D 乐器模型（钢琴）上进行播放。

各个主要模块下，还有具体实现某一功能的小模块，系统整体结构如图 3-1 所示：

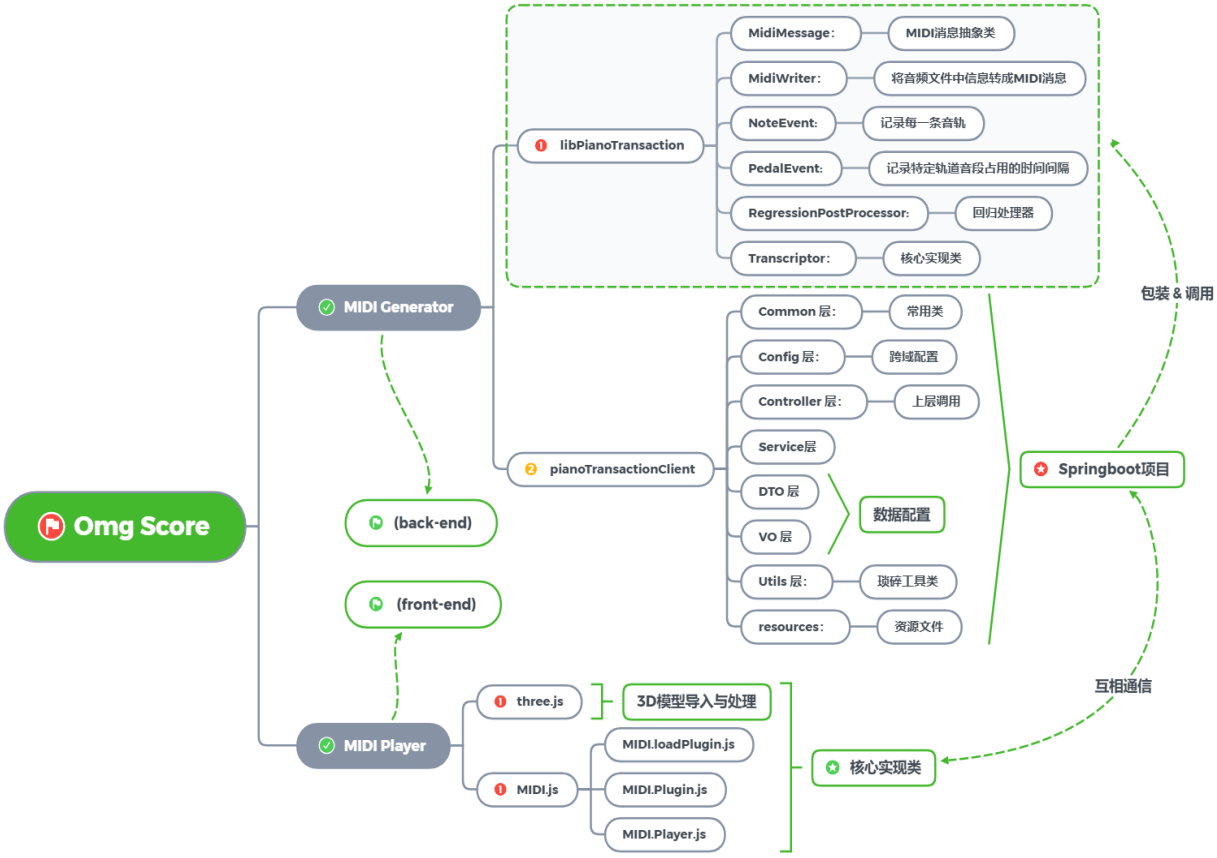


图 3-1 系统整体结构图设计

3.1.1 MIDI Generator

MIDI Generator 模块，即本项目的服务端是通过 java 语言，使用 Apache Maven 作为软件项目管理及自动构建工具，利用 springboot 后端框架实现。具体需将项目划分为两个子模块：libPianoTransacion 和 pianoTransactionClient。

- libPianoTransacion 模块：该模块用来实现实现音频文件比如 mp3 文件到 MIDI 文件的转换逻辑，主要分为 MidiMessage、 MidiWriter、NoteEvent、PedalEvent、RegressionPostProcessor、Transcriptor 六个类。
- pianoTransactionClient 模块：该模块其实是一个完整的依托于 springboot 框架的后端项目，用来包装 libPianoTransaction 模块，并实现服务端整体功能：mp3 等音频文件的导入与转储、mp3 到 MIDI 文件的转化（包装前一模块）、MIDI 文件的存储、预处理和回传。

3.1.2 MIDI Player

MIDI Player 模块，即本项目的客户端主要通过 Html、Css、Javascript 实现，使用 Reat 框架和重要的两项技术：three.js 和 MIDI.js。

其中，前者负责导入立体模型并展示动态界面，后者负责具体 MIDI 文件的解析。具体实现了三个功能模块：界面展示模块、MIDI 文件播放模块、文件传输模块。

- 界面展示模块：该模块主要分为基本组件的展示以及 3D 图形展示两个部分：
  - 基本组件的展示：包括最基本的文件上传下载组件、项目信息显示组件、3D 乐器模型自定义调节组件；
  - 3D 图形展示：包括 3D 乐器模型（本应用中使用钢琴模型）的导入、位姿调节、形态变换的实现；
- MIDI 文件播放模块：该模块用来实现 MIDI 文件的解析，并将其处理成可以作为电子音乐播放的形式；
- 文件传输模块：该模块用于前后端的通信，主要实现音频文件的上传以及接收到的 MIDI 文件的下载和保存。

3.2 对象模型和关键数据结构设计

3.2.1 MIDI Generator

1. libPianoTransacion

由上文得到，本模块分为 6 个子模块，具体 UML 类图如图 3-2 所示。

- **MidiMessage 类：**
  - 功能：将 MIDI 消息具体抽象成的一个类，便于文件的写入。
  - 属性：
    - private final float a：时间
    - private final float b：音调或控制改变
    - private final float c：播放速度
    - private final int type：类型
  - 方法：包括构造函数和 Get 方法。
- **MidiWriter 类：**
  - 功能：提供将 mp3 文件中提取的信息转化为不同的 MidiMessage，并进行一定程度的封装，主要功能就在于将 MP3 文件经过转化后得到的数据以字节流的形式进行存储并作为返回值返回,这是 MIDI 文件在计算机中存储的一种数据形式，此外也完成了将 MIDI 类型的文件的字节流写入提供的输出流。
  - 属性：无。
  - 方法：只有一个方法 writeEventsToMidi，用于将字节流数组写入输出流数组。
- **NoteEvent 类：**
  - 功能：用来记录每一条轨道相关的数据信息，包括起始和终止时间，有关的 midi 记录以及播放的速度等属性。
  - 属性：包括 onsetTime、offsetTime、midiNote 和 velocity。
    - onsetTim / offsetTime：记录计时的开始时刻和结束时刻；
    - midiNote：记录 midi 文件的特征值；
    - velocity：记录 midi 文件相邻音符之间转换的速度。
  - 方法：包括构造函数和 Get 方法。
- **PedalEvent 类：**
  - 功能：用于记录特定轨道音段所占用的时间间隔。
  - 属性：包括 onsetTime 和 offsetTime,用于记录计时的开始时刻和结束时刻。
  - 方法：包括构造函数和 Get 方法。
- **RegressionPostProcessor 类：**
  - 功能：回归处理器。用于实现 MIDI 文件的各种特征的改变和赋值给外部变量。
  - 属性：
    - frames_per_second：帧频

- classes_num：音轨数
- onset_threshold：发音的阈值
- offset_threshold：偏音的阈值
- frame_threshold：帧频的阈值
- pedal_offset_threshold：持续的偏音的阈值
- begin_note：最开始的音调
- velocity：播放速度

❖ 方法：

- Public RegressionPostProcessor(int frames_per_second, int classes_num, float onset_threshold, float offset_threshold, float frame_threshold, float pedal_offset_threshold)：构造函数。
- private float[ ][ ] read2DArray(float[ ][ ] x, int start1, int end1, int start2, int end2)：将 x 数组中第一维坐标属于[start1,end1),第二维坐标属于[start2,end2)的元素赋值给新的二维数组，并返回这个新的二维数组。
- private void write2DArray(float[ ][ ] source, float[ ][ ] des, int start1, int end1, int start2, int end2)：将 source 中第一维坐标属于[0,end1-start1),第二维坐标属于[0,end2-start2)的元素赋值给 des 中第一维坐标属于[start1,end1),第二维坐标属于[start2,end2)的元素。
- public List<Object> outputDictToMidiEvents(Map<String, float[ ][ ]> dict)：创建 ArrayList<>型变量 output，令变量 out1 = outputDictToNotePedalArrays(dict)，est_on_off_note_vels = out1.get(0)，est_pedal_on_offs = out1.get(1)，output 中加入 detectedNotesToEvents(est_on_off_note_vels)，根据 est_pedal_on_offs.size()是否不为 0 来决定是否将 detectedPedalsToEvents(est_pedal_on_offs) 加入 output 中。
- private List<NoteEvent> detectedNotesToEvents(List<float[ ]> est_on_off_note_vels)：将 est_on_off_note_vels 中的各个 float 数组的第 1 个和第 2 个元素作为参数使用构造函数生成新的 PedalEvent 型对象，将该对象加入 output 数组中。
- private List<List<float[ ]> outputDictToNotePedalArrays(Map<String, float[ ][ ]> dict)：创建 List<List<float[ ]>型变量 output，在 output 中加入 outputDictToDetectedNotes(dict)，根据"reg_pedal_offset_output"是否存在于 dict 的键中，来决定是否在 output 中加入 outputDictToDetectedPedals(dict)。

- private List<float[ ][ ]> getBinarizedOutputFromRegression(float[ ][ ] reg_output, float threshold, int neighbour)：从回归结果中获得二值化的输出。
- private boolean is_monotonic_neighbour(float[ ] x, int n, int neighbour)：检测是否是递增或递减的相邻项
- private List<float[ ]> outputDictToDetectedNotes(Map<String, float[ ][ ]> dict)：获取每一个非持续音的开始时间和结束时间。
- private List<float[ ]> outputDictToDetectedPedals(Map<String, float[ ][ ]> dict)：获取每一个持续音的开始时间和结束时间。
- private float[ ] extract2DElementsOf2DArray(float[ ][ ] source, int index2d)：创建 float[ ]型变量 output，将 source[i][index2d]赋值给 output[i]，返回 output。
- private float[ ][ ] listToArray(List<float[ ]> l)：将 l 赋值给二维数组，返回该二维数组。
- private float[ ] listToArray2(List<Float> l)：将 l 赋值给一维数组，返回该二维数组。
- private List<float[ ]> stack4ArraysToList(float[ ] a, float[ ] b, float[ ] c, float[ ] d)：将 a[i]，b[i]，c[i]，d[i]作为新的 float 数组，加入到 List<float[ ]>型变量 output 中，返回 output。
- private List<float[ ]> stack2ArraysToList(float[ ] a, float[ ] b)：将 a[i]，b[i]作为新的 float 数组，加入到 List<float[ ]>型变量 output 中，返回 output。
- private float[ ] add2ArraysAndDivide(float[ ] a, float[ ] b, int divide)：创建 float[ ]型变量 output，output[i] = (a[i]+b[i])/divide。
- private List<float[ ]> noteDetectionWithOnsetOffsetRegress(float[ ] frame_outputs, float[ ] onset_outputs, float[ ] onset_shift_outputs, float[ ] offset_outputs, float[ ] offset_shift_outputs, float[ ] velocity_outputs, float frame_threshold)：创建 ArrayList<float[ ]>型变量 output_tuples，根据各参数的取值情形，生成对应的 float[ ]型数组，目的是使用 onset 和 offset 做回归运算来检测音调。
- private List<float[ ]> pedalDetectionWithOnsetOffsetRegress(float[ ] frame_outputs, float[ ] offset_outputs, float[ ] offset_shift_outputs, double frame_threshold)：创建 ArrayList<float[ ]>型变量 output，根据各参数的取值情形，生成对应的 float[ ]型数组，目的是使用 onset 和 offset 做回归运算来检测持续音。

- **Transcriptor 类：**

- 功能：将 MP3 文件分解为不同的轨道记录相关数据，转化为可用于 MIDI 类型文件存储的信息流，是实现 MP3 文件转化为 MIDI 文件的核心实现类。
- 属性：
  - private final OrtEnvironment _env：onnx 运行时系统的主机对
  - private final OrtSession _session：_env 的特定型号
  - private final int segment_samples = 16000 * 10：段的样本值
  - private final int frames_per_second = 100：帧频
  - private final int classes_num = 88：音轨数
  - private final float onset_threshold = 0.3f：起始阈值
  - private final float offset_threshold = 0.3f：终止阈值
  - private final float frame_threshold = 0.1f：帧阈值
  - private final float pedal_offset_threshold = 0.2f：持续音偏移阈值
- 方法：
  - public Transcriptor(String modulePath)：调用构造函数初始化_env 和 _session
  - public byte[ ] transcript(float[ ] pcm_data)：将 pcm_data 数组分段存入 byte[ ]中。
  - private List<float[ ]> enframe(float[ ] data, int segment_samples)：将 data 数组按照每 segment_samples 个元素一组，赋值给 batch，并且是一半冗余地赋值，即若 data 为 1 2 3 4 5 6 7 8 且若 segment_samples = 4, 则传为 1 2 3 4; 3 4 5 6; 5 6 7 8。
  - private Map<String, List<float[ ][ ]>> forward(List<float[ ]> data)：建立"reg_onset_output"、"reg_offset_output"、"frame_output"、"velocity_output"、"reg_pedal_onset_output"、"reg_pedal_offset_output"、 "pedal_frame_output"这些键到值的映射，调用 onnx 文件生成对应的 MIDI 流并保存到映射中。
  - private float[ ][ ] deframe(List<float[ ][ ]> x)：通过使用参数将 x 的元素值进行数学运算来调整。
  - private void write2DArray(float[ ][ ] source, float[ ][ ] des, int start1, int end1, int start2, int end2)：把 source 的 0 到 end1 - start1-1 行，0 到 end2 - start2-1 列的元素赋值给 des 的 start1 到 end1-1 行，start2 到 end2-1 列的元素。
  - private float[ ][ ] read2DArray(float[ ][ ] x, int start1, int end1, int start2, int end2)：读数组，与 write2DArray 相反的操作。

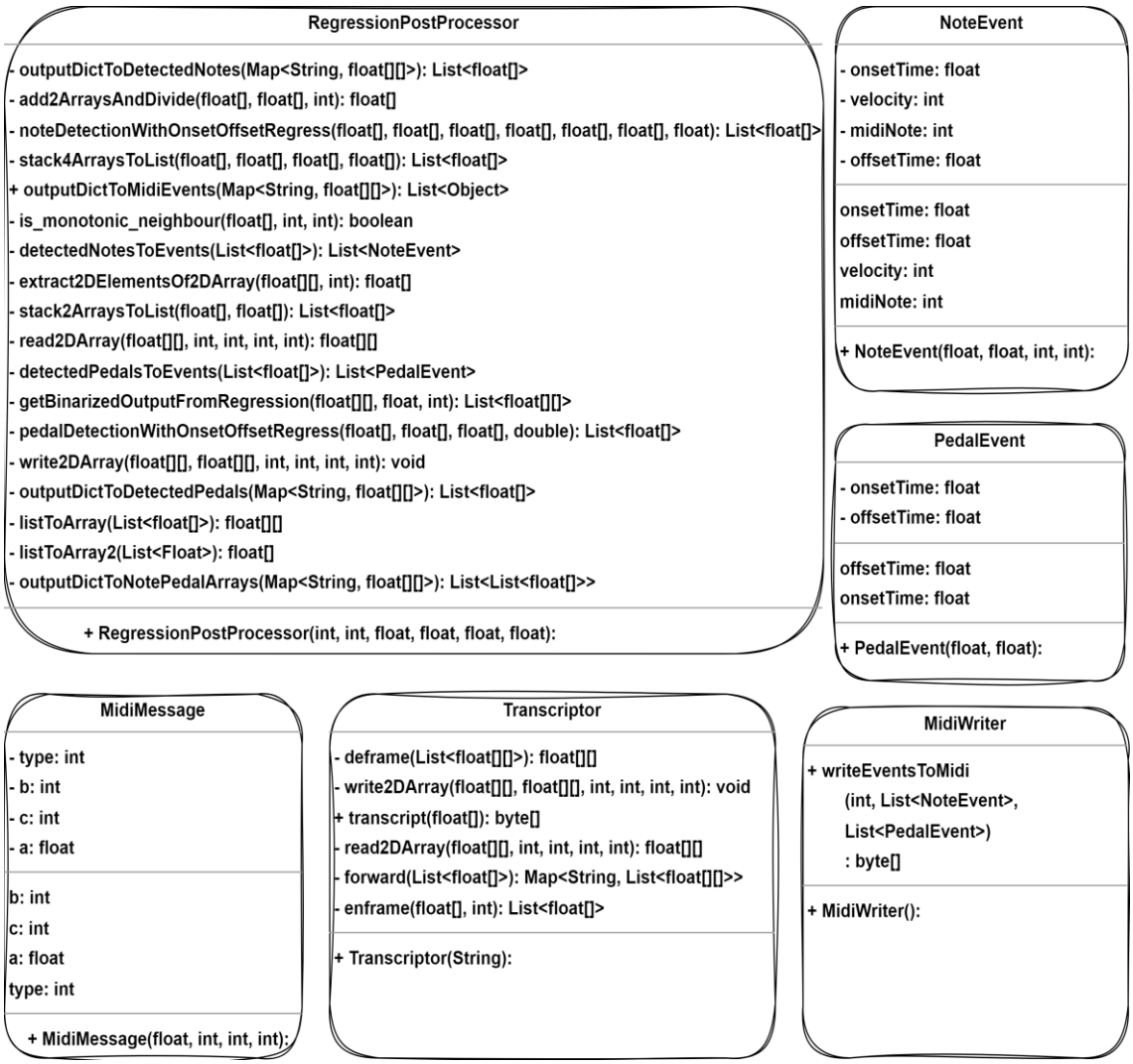


图 3-2 libPianoTransacion 类图

2. pianoTransactionClient

使用 springboot 框架，进行前后端通信，使用上个模块中包装好的 transcriptor 类，获取客户端传输的 mp3 文件，同时进行转换并将转换好的 MIDI 文件返回客户端。

整体 UML 类图如图 3-3 所示。

并且关于这一模块，我们按照后端项目的基本构架分为了 Common 层、Config 层、Controller 层、DTO 层、Service 层、Utils 层、VO 层，以及启动类 TranscriptionApplication 和资源文件等。

作为轻量级项目，本应用不涉及数据库，所以去掉了相应的 Model 层以及 DAO 层。下面进行具体的阐述：

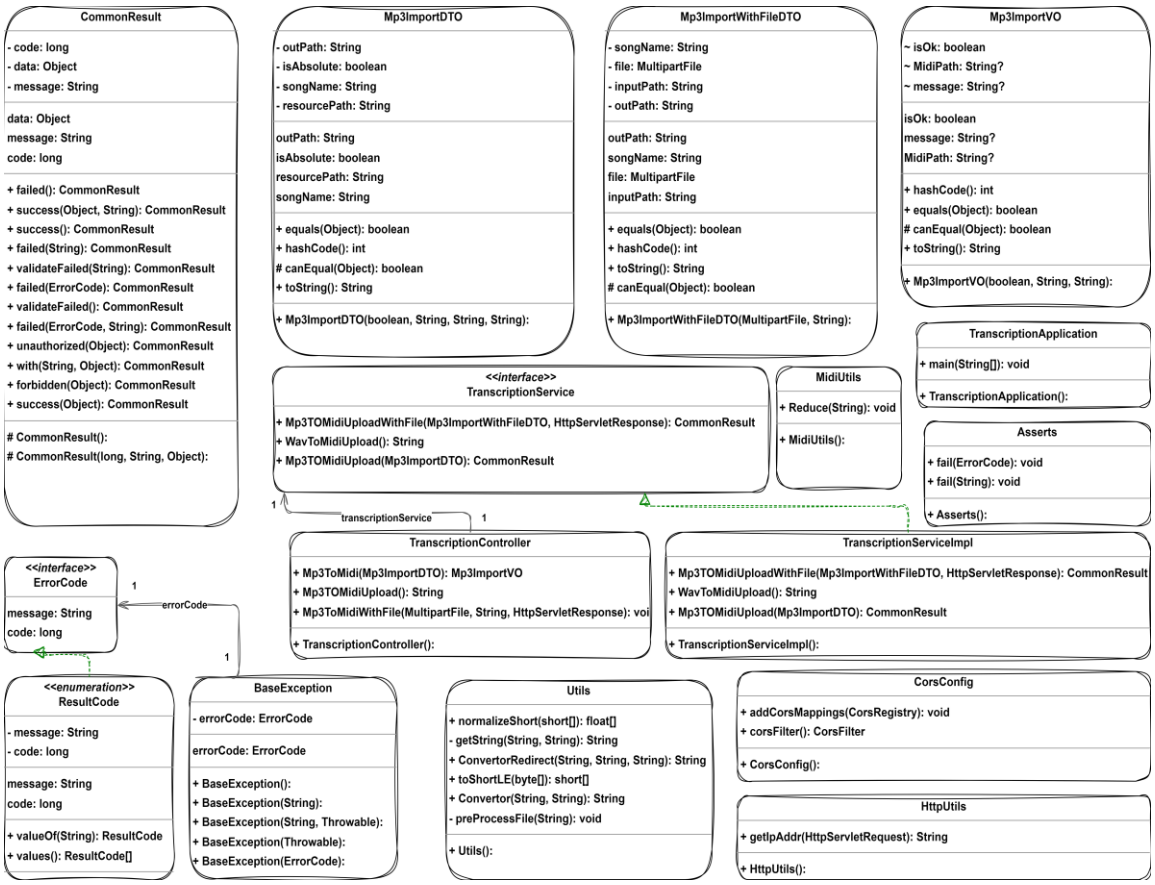


图 3-3 MIDI Player 类图

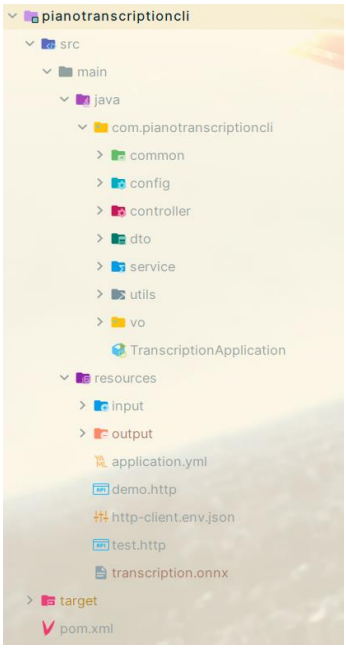


图 3-4 Player 项目架构

本项目结构如图 3-4 所示。

- Common 层：主要实现一些基本常量的定义以及通用接口和类，这里分为 api 和 exception 两部分。

❖ api 部分规定了自己设计的一些异常码（枚举类）诸如 SUCCESS、FAILED、UNAUTHORIZED，以及与之相关的返回 api 接口和类型；

❖ exception 部分规定了自己设计的一些异常包装类；

- Config 层：暂时只规定了 CorsConfig 类，继承了 WebMvcConfigurer 接口，给项目提供了跨域支持；

- Controller 层：简洁起见，只设计了一个 TranscriptionController 类，负责请求转发，接受页面过来的参数，传给 Service 层处理，接到返回值，再传给

页面。controller 层负责具体的业务模块流程的控制，在此层要调用 service 层的接口来控制业务流程，控制的配置也同样是在 Spring 的配置文件里进行，针对具体的业务流程，会有不同的控制器。我们具体的设计过程可以将流程进行抽象归纳，设计出可以重复利用的子单元流程模块。这样不仅使程序结构变得清晰，也大大减少了代码量。

- Service 层：主要负责业务模块的应用逻辑应用设计。同样是首先设计接口，再设计其实现类，接着再 Spring 的配置文件中配置其实现的关联。这样我们就可以在应用中调用 Service 接口来进行业务处理。Service 层的业务实，具体要调用已经定义的 dao/DTO 层接口，封装 Service 层业务逻辑有利于通用的业务逻辑的独立性和重复利用性。程序显得非常简洁。
  这里同样也只写了一个接口 TranscriptionService 以及其继承覆写类 TranscriptionServiceImpl，规定了具体的业务函数 Mp3TOMidiUploadWithFile(Mp3ImportWithFileDTO, HttpServletResponse): CommonResult，以及 Mp3TOMidiUpload(Mp3ImportDTO): CommonResult 等，主要就是对 libPianoTransacion 模块的函数进行包装和调用，从而实现音频文件转为 MIDI 文件的业务逻辑。

- Utils 层：广泛意义上的工具层，定义了一些零碎的功能类：

  - HttpUtils 类：定义了从客户端拿到用户 IP 的方法：public static String getIpAddr(HttpServletRequest request)；

  - MidiUtils 类：定义了读取并播放 MIDI 文件的方法：public static void Reduce(String output)；

  - Utils 类：定义了调用 libPianoTransacion 模块函 libPianoTransacion 数的具体方法，用来实现 MIDI 文件的转化和预处理，并打印日志，具体分为：

    - public static short[ ] toShortLE(byte[ ] bytes)；用来将字流转化为 short 类型数组；

    - public static float[ ] normalizeShort(short[ ] shorts)；用来实现数组规范化和预处理；

    - public static String Convertor(String resourcePath, String songName) throws Exception；调用 preProcessFile 函数，实现服务端本地的音频文件到 MIDI 文件的转化；

- private static String getString(String resourcePath, String outPutFilePath) throws IOException, OrtException, InvalidMidiDataException；字符串分割、处理方法；
- public static String ConvertorRedirect(String resourcePath, String songName, String outputPath) throws Exception；用于重定向输出文件位置的音频文件至 MIDI 文件的转化方法；
- private static void preProcessFile(String fileName) throws Exception；这是具体的文件转化方法，调用了 libPianoTransacion 中的具体方法、ffmpeg 转化、以及资源文件中的组件；

- DTO (Data Transfer Object) 层：数据传输对象，即在应用程序不同 tie(关系)之间传输的对象。主要作为 Controller 层和 Service 层（主要是前者）基本函数的传入参数，这里设计了 Mp3ImportDTO 和 Mp3ImportWithFileDTO 两个基本类，用于和文件传输和转换的相关逻辑调用。
- VO (value object) 层：规定了值对象，通常用于业务层之间的数据传递，和 PO 一样也是仅仅包含数据而已。但应是抽象出的业务对象，可以和表对应，也可以根据业务的需要。其实同 DTO(数据传输对象)，在 web 上传递。这里主要用于包装 Controller 层和 Service 层的返回值。

3.2.2 MIDI Player

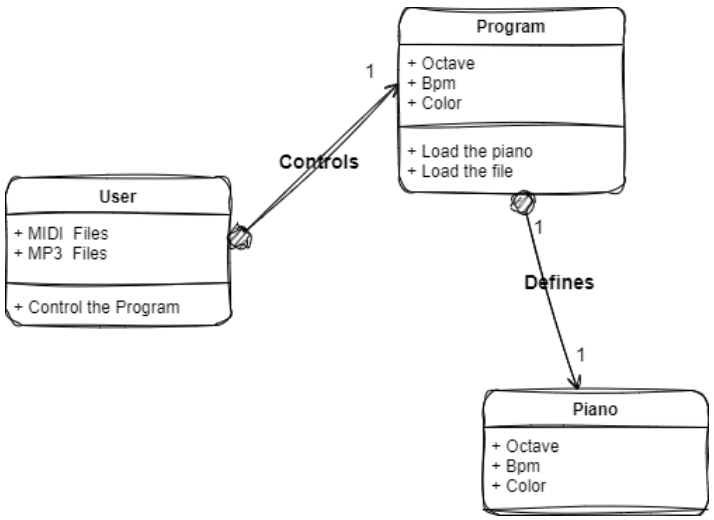


图 3-5 MIDI Players 的 UML 图

1. three.js

three.js是一个让web GL更加容易实现的JavaScript库，通过它实现3D效果可以简化代码的复杂度。Three.js作为单个JavaScript文件，可以在没有Web服务器的

情况下在本地工作。

在index.html文件中，我们对Three.js进行了运用：

首先，钢琴是保存在一个名为piano.dae的文件里，需要使用loarder.load对其进行引用。基础场景为一个平坦的平面THREE.PlaneGeometry，并定义其大小和材质、平面于钢琴的夹角（默认为垂直）、平面的纵坐标。接下来我们需要一组平行光THREE.DirectionalLight，并设置其亮度为0xffffff，以及光源的位置、指向、影子的深度。

以上两个部分是非常重要的，我们需要使用scene.add（）函数将它们加载到场景中去。

2. MIDI.js

这是一个用来解析MIDI文件的模块。软件将用户选择好的MIDI文件进行读取，其中存储了歌曲的速度、时常、每个音符的时值等信息，并把这些信息传到虚拟钢琴上，就可以使歌曲播放出来。

- MIDI.loadPlugin.js - 决定最好用哪个框架
- MIDI.Plugin.js - 控制 MIDI 输出
- MIDI.Player.js - 播放 MIDI 流

3.3 关键算法设计

基于识谱演奏一体化的音乐教学播放应用主要由两个模块组成，下面分别分析服务端（后端）模块和客户端（前端）模块的主要算法设计：

3.3.1 服务端模块

首先不妨介绍一下 MIDI 文件在 C++中的处理。

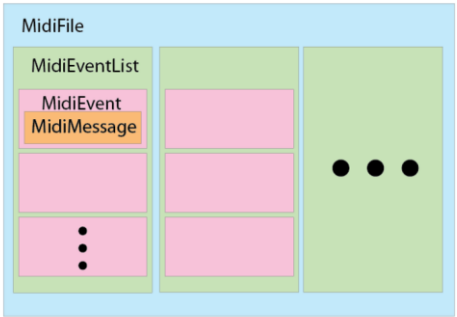


图 3-6 MIDI 文件的结构划分

图 3-2 是 MIDI 文件的结构划分，这里参考了一个使用 C++编写的用于读写标准 MIDI 文件的类库（https://github.com/craigsapp/midifile）。自上而下分析，MidiFile 类显示为 MidiEvents 的二维数组，其中第一维是轨道列表，第二维是 MidiEvents 列表；MidiEventsList 是管理 MIDI 文件轨道的 MidiEvents 列表的数据结构；MidiEvents 是 MidiFile 中 MidiMessages 的主要存储单元。该类由一个刻度时间戳（增量或绝对）和一个 MIDI 消息字节向量（或标准 MIDI 文件元消息）组成。而 MidiMessage 就是 MidiEvents 的基类。这是代表 MIDI（或元）消息的无符号字节的向量。

main 函数中的算法的流程如下图所示：

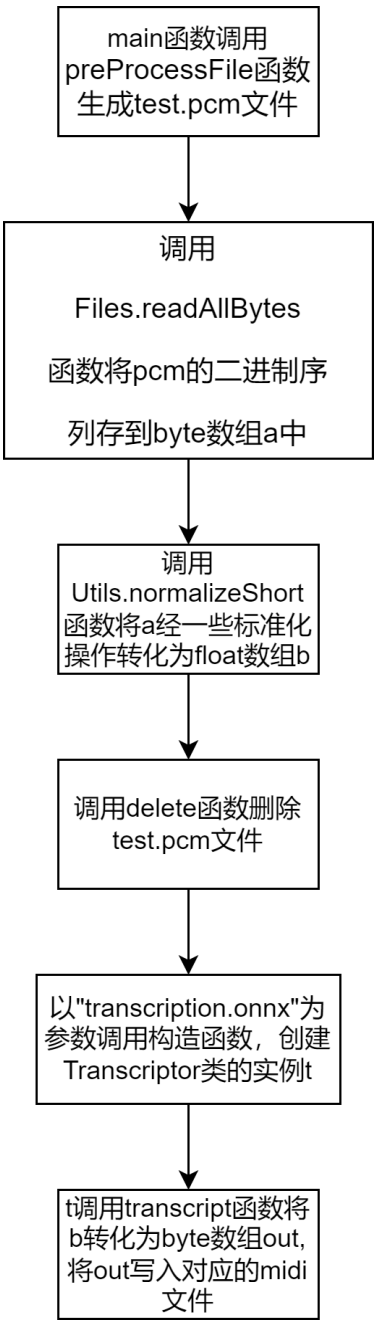


图 3-7 main 函数中的算法的流程

transcript 函数中算法的流程如下图所示：

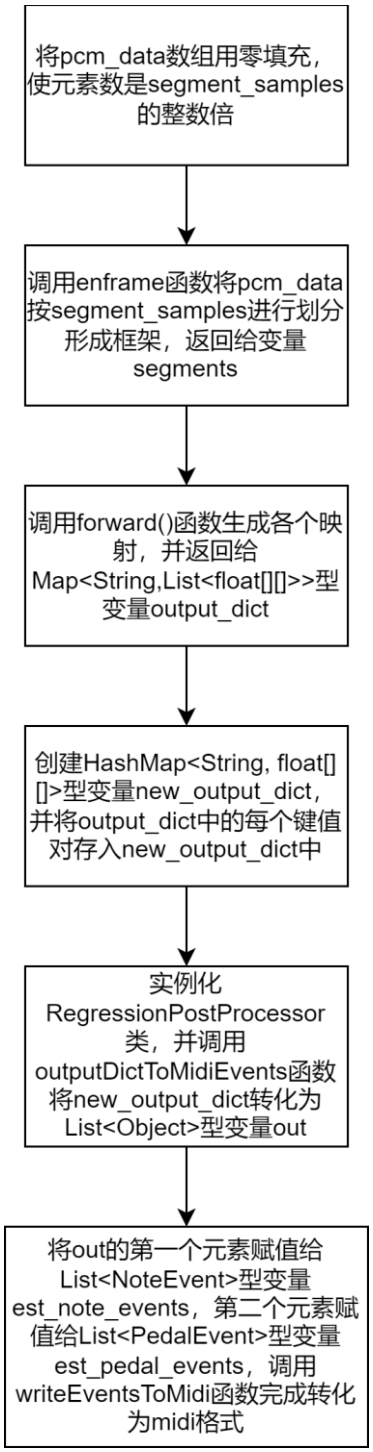


图 3-8 transcript 函数中算法的流程

3.3.2 客户端模块

```
floor = new THREE.Mesh(new THREE.PlaneGeometry(8000, 8000), new THREE.MeshBasicMaterial({ color: 0x00f5ff }));
floor.rotation.x = - 90 * (Math.PI / 180); //桌面与钢琴的夹角，默认为垂直
floor.position.y = -0.45; //桌面的纵坐标
floor.receiveShadow = true; //接受钢琴投影在桌面上
scene.add(floor); //添加到场景中
```

图 3-9 桌面设计

- 设置桌面为大小为 8000x8000 的平面，第二个参数是它的材质，其中颜色为 0x00f5ff。
- rotation 是平面和虚拟钢琴的夹角，默认为垂直，单位为弧度，因此需要将角度设为-90 度，也就是 -90 * (pi / 180)
- position 是平面的高度，-0.45 是使钢琴下表面恰好接触桌面的数值。
- 最后需要将桌面添加到场景中。

```
var spotlight = new THREE.DirectionalLight(0xffffff); //平行光强度
spotlight.position.set(1.0, 2.4, -7.5); //光源位置
spotlight.target.position.set(6.0, -6, 7); //光源指向
spotlight.shadowDarkness = 0.75; //影子深度
spotlight.intensity = 1; //光照强度
spotlight.castShadow = true; //是否产生阴影
//决定有多少像素用来构成阴影
spotlight.shadowMapWidth = 2048;
spotlight.shadowMapHeight = 2048;
//投影相机
spotlight.shadowCameraNear = 5.0; //表示到距离光源的哪一个位置开始生成阴影
spotlight.shadowCameraFar = 20.0; //表示到距离光源的哪一个位置可以生成阴影
spotlight.shadowBias = 0.0025; //解决自遮挡阴影瑕疵(shadow acne)
//投影边界
spotlight.shadowCameraLeft = -8.85;
spotlight.shadowCameraRight = 5.5;
spotlight.shadowCameraTop = 4;
spotlight.shadowCameraBottom = 0;
scene.add(spotlight);
var light = new THREE.DirectionalLight(0xddffff, 0.5);
light.position.set(1, 1, 1).normalize();
scene.add(light);
var light = new THREE.DirectionalLight(0xff5555, 0.5);
light.position.set(-1, -1, -1).normalize();
scene.add(light);
```

图 3-10 光源设计

- DirectionalLight 是平行光。设置光源位置、光源指向。
- shadowCameraNear 表示从距离光源的哪一个位置开始产生阴影。
- shadowCameraFar 表示最远到距离光源的哪一个位置可以产生阴影。

最后把添加的所有平行光添加到场景中去。

3.4 数据管理说明

omg-score 项目借助 web 平台实现，整体的的数据被分为两个部分：本地缓存以及云端资源。云端资源主要存储本地上传的 MP3 音频文件比如 MP3 文件，以及换成的 MIDI 文件；本地缓存，即客户端本地，存储 MP3 文件和 MIDI 文件。

3.4.1 云端资源

本 web 项目使用阿里云的云 4 核 8G 服务器以及相应的云端端源。

- 首先，项目服务端运行在云服务器上，服务器操作系统为 Ubuntu20.04。
- 相应地，云端资源还包括客户端上传的 MP3 文件，以及转化后即将返回客户端的 MIDI 件。

3.4.2 本地缓存

- 本地缓存首先存储用户（客户端）将要上传给服务端的 MP3 文件，以及服务端返回的 MIDI 文件；
- 同时，本地缓存存储了客户端运行的相关日志文件，同时设置了按照天数更新并自动清理的策略，保证不占用太多本地磁盘空间。

3.4.3 数据读取使用

就本项目而言，用户首先在客户端首先向服务端上传自己本地缓存的 MP3 文件；转化处理后，服务端再将 MIDI 文件返回，由用户在客户端自行下载并选择存储。用户在客户端可以选择只播放 MIDI 文件对应的乐曲，也可以选择下载 MIDI 文件。

同时，在存储时，云端数据以及本地数据其实是自动同步更新的，尤其本地缓存的日志文件设置了自动清理更新机制。在本地缓存中选择替换旧数据的方法，而在云端资源中选择添加的方法。故可分别产生近期数据与全部数据，方便用户的使用。

4 实现与测试

4.1 实现环境与代码管理

4.1.1 应用实现环境

本项目分为客户端 MIDI Player 和服务端 MIDI Generator。

- 客户端编程语言为 Javascript，Html，CSS，以及些许 Vue 和 Tapescript，开发 IDE 使用 Visual Studio Code；
- 服务端为典型的使用 Springboot 框架的 Java web 项目，编程语言为 Java 以及 Python，开发 IDE 使用 IntelliJ IDEA Ultimate。

基于 web 项目的特性，开发者的软硬件环境对软件开发不构成影响，故此处不进行说明。

4.1.2 代码管理

本项目使用 Git 作为版本控制软件，使用 GitHub 为在线软件源代码托管服务平台，代码仓库地址为：https://github.com/SleepyLGod/omg-score。

下面将从**提交记录展示、总体分析、贡献度统计**三方面描述：

（一）Git 提交记录（部分）图如图 4-1 所示。

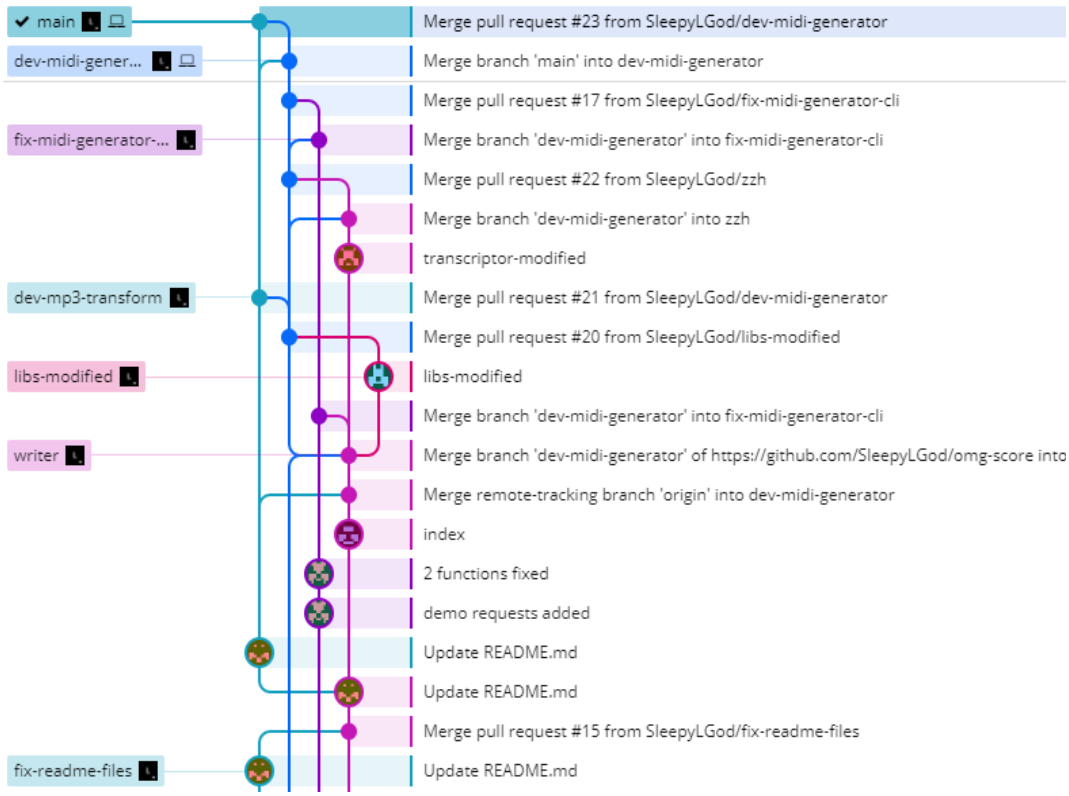


图4-1 代码仓库更新记录(部分)

（二）总体来讲，开发过程中，通过 Git log 我们得到：

- 总共提交 **99** 次；
- 贡献 ID 为 **8** 个（由于平台不同，一人可有多个 ID）；
- 由于项目特性，总修改代码行数为 **167591** 行；
- 提出一条始终 open 的 issue，供拓展功能的实现；
- 提出 **20** 条 pull request 并 merge 了 **15** 条，如图 1-2 所示；

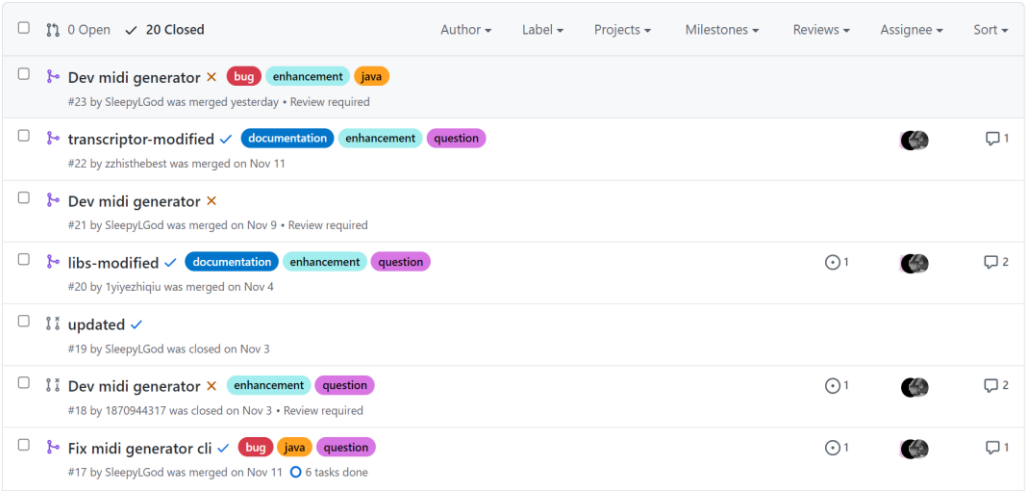


图4-2 仓库PR记录(部分)

（三）关于 GitHub 的 username 以及对应的贡献度：

- 路昊东同学 SleepyLGod（提交用户 ID 为 lhd / Haodong Lu / odin / dong）：
  总共提交 61 次，增加代码/文档共 39030 行，删除 23151 行；
- 吴潇同学 1870944317（提交用户 ID 为 1870944317）：总共提交 4 次，增加代码/文档共 149443 行，删除 37123 行；
- 薛江博同学 1yiyezhiqiu（提交用户 ID 为 1yiyezhiqiu）：总共提交 1 次，增加代码/文档共 65 行，删除 23 行。
- 张子豪同学 zzhisthebes（提交用户 ID 为 zzhisthebes）：总共提交 1 次，增加代码/文档共 34 行，删除 21 行；
- dependbot 是使用 GitHub Action 等自动设置的安全监测 bot。

四人代码贡献度统计在 git log 中如图 1-3 所示，在平台上如图 4-4 所示。

```
1870944317      added lines: 149443, removed lines: 37123, total lines: 112320
1yiyezhiqiu     added lines: 65, removed lines: 23, total lines: 42
dependabot[bot] added lines: , removed lines: , total lines:
dong    added lines: 802, removed lines: 207, total lines: 595
Haodong Lu      added lines: 288, removed lines: 202, total lines: 86
lhd     added lines: 15945, removed lines: 661, total lines: 15284
odin    added lines: 11070, removed lines: 252, total lines: 10818
zzhisthebest    added lines: 34, removed lines: 21, total lines: 13
```

图 4-3 git log 代码贡献度统计

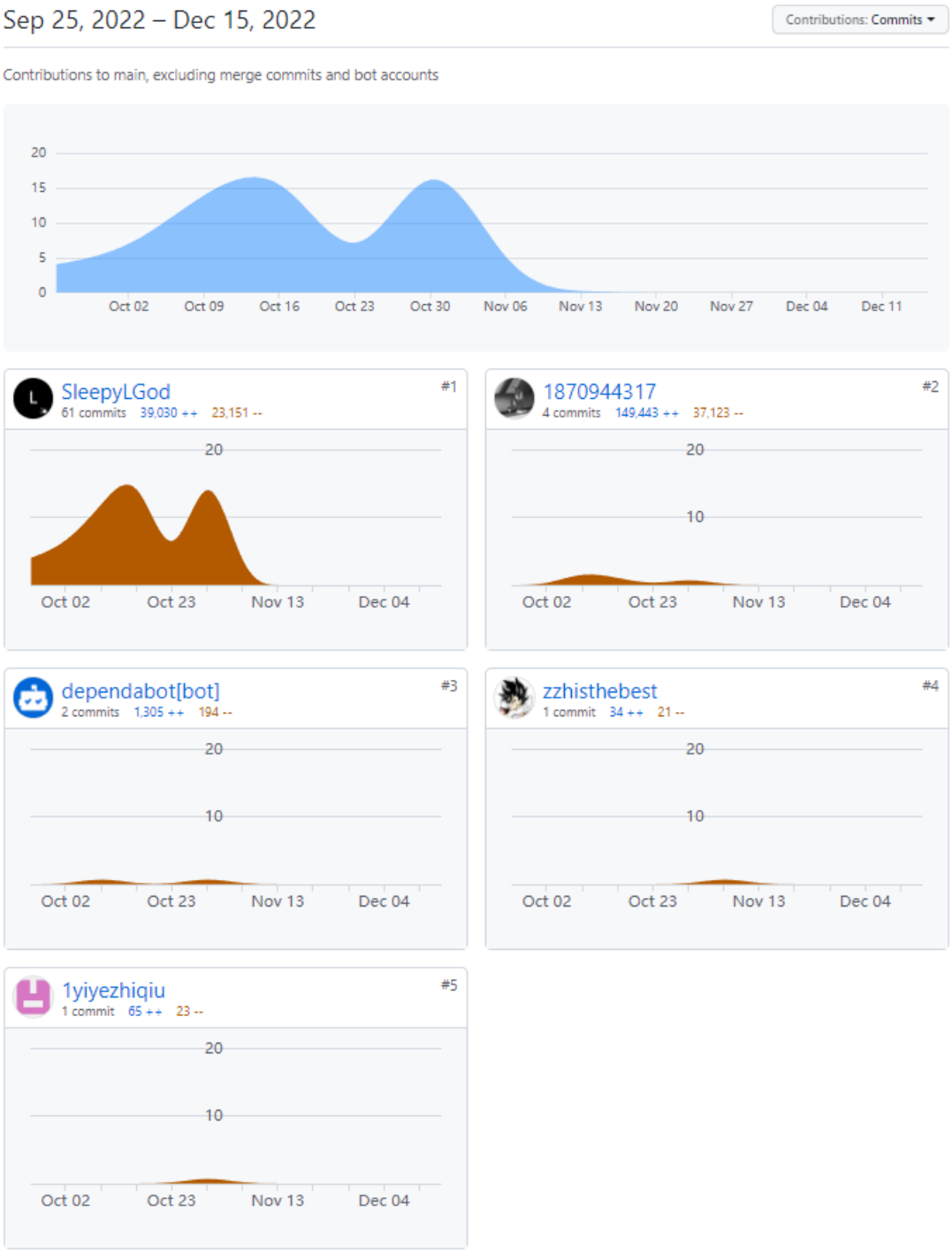


图4-4 GitHub平台代码贡献度统计

4.2 关键函数说明

1. MidiWriter：

主要功能在于将 MP3 文件经过转化后得到的数据以字节流的形式进行存储并作为返回值返回,这是 MIDI 文件在计算机中存储的一种数据形式，同时将

MIDI 类型的文件的字节流写入提供的输出流。基本的 get、set 方法、等均在 3.2 对象模型和关键数据结构设计中指出，这里描述的是在实现流程（此次可见 3.3 关键算法设计中的流程图）中一些关键函数的调用。

首先设置每节拍滴答频率和每秒节拍数，然后创建每节拍 384（每节拍滴答频率） tick 的 midi 序列，在该 midi 序列下创建一条轨道 t 并设定相应节奏，待后续创建一系列的 MidiEvent 对象并设置好相应参数后，将它们加入到该轨道内，然后调用 sort 方法根据每个对象的 OnsetTime 即开始时刻对 roll 中加入的几个 MidiMessage 对象进行排序，这样在该 midi 序列下，一个完整的轨道就建成了。注意每一个 midi 乐谱都包含很多个音轨，所以在输出的 midi 文件在此记录存储时需要多次调用该方法，每次建立一个新的轨道进行存储。

2. Transcriptor：

将 MP3 文件分解为不同的轨道记录相关数据，转化为可用于 MIDI 类型文存储的信息流。

首先将 pcm_data 数组用 0 填充，使得元素数是 segment_samles 的整数倍，目的在于复制指定的数组，用零截断或填充，以便副本具有指定的长度。
对于在原始数组和副本中都有效的所有索引，这两个数组将包含相同的值。
对于在副本中有效但不是原始索引的任何索引，副本将包含 0f。接着再调用 enframe 函数，将 pcm_data 按照 segment_samles 进行划分形成框架，返回给变量 segments。再调用 forward()函数，生成 MP3 到 MIDI 的各个映射，并返回给 Map<String, List<float[ ][ ]>> 型变量 output_dict, 创建 HashMap<String, List<float[ ][ ]>>型变量 new_output_dict，并将 output_dict 中的每个键值对存入 new_output_dict 中。

这些工作完成后，对 RegressionPostProcessor 类进行实例化，该类用于实现 MIDI 文件的各种特征的改变和赋值给外部变量。调用 outputDictToMidiEvent 将 new_output_dict 转化为 List<Object>型变量 out，再将 out 的第一个元素赋值给 List<NoteEvent>型变量 est_note_events，第二个元素赋值给 List<PedalEvent>型变量 est_note_events，最后调用 writeEventsToMidi 函数完成 midi 格式的转化。

4.3 测试计划和测试用例

软件测试常用的方法有：静态测试、动态测试、黑盒测试、白盒测试。

静态测试法：静态测试方法指软件代码的静态分析测验，此类过程中应用数据较少，主要过程为通过软件的静态性测试(即人工推断或计算机辅助测试)测试程序中运算方式、算法的正确性，进而完成测试过程。

动态测试法：计算机动态测试的主要目的为检测软件运行中出现的问题，较静态测试方式相比，其被称为动态的原因即为其测试方式主要依赖程序的运用，主要为检测软件中动态行为是否缺失、软件运行效果是否良好。

黑盒测试法：黑盒测试，顾名思义即为将软件测试环境模拟为不可见的“黑盒”。通过数据输入观察数据输出，检查软件内部功能是否正常。测试展开时，数据输入软件中，等待数据输出。数据输出时若与预计数据一致，则证明该软件通过测试，若数据与预计数据有出入，即便出入较小亦证明软件程序内部出现问题，需尽快解决。

白盒测试法：白盒测试相对于黑盒测试而言具有一定透明性，原理为根据软件内部应用、源代码等对产品内部工作过程进行调试。测试过程中常将其与软件内部结构协同展开分析，最大优点即为其能够有效解决软件内部应用程序出现的问题，测试过程中常将其与黑盒测试方式结合，当测试软件功能较多时，白盒测试法亦可对此类情况展开有效调试。

由于Web网页应用的特点，我们难以对各个函数进行逐一测试，因此我们采用 黑盒测试的方法，去测试每个模块的功能是否能够正确实现预期目标。

1. Web网页应用的加载

Web网页应用的加载的测试样例与预期测试结果如表 4-1 所示。

表 4-1 Web网页主界面加载测试表

测试样例	预期结果
用户打开web应用的网站，向服务器申请该3D乐器演奏网页	经过一段时间的加载，待Soundfont和Web Audio API加载完成后完成主界面的展示
用户用鼠标拉动3D乐器即可改变其放置位置，360度方向上都能旋转	旋转放置3D乐器在一个任意的角度，且不会影响各个功能的使用

最后加载得到的实际结果如下图4-5所示。



图 4-5 Web 网页主界面加载结果

实际的旋转放置 3D 乐器的结果如下图 4-6 所示。



图4-6 旋转放置3D乐器展示结果

2. 控件中各个功能模块的实现

主界面内控件的各个功能模块实现的测试样例和预期测试结果如表4-2所示。

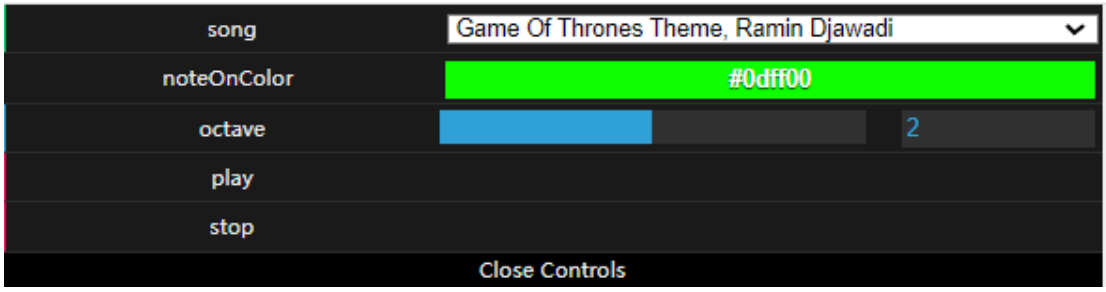
表4-2控件中各个功能模块测试表

测试样例	预期结果	实际结果
用户在song模块下从提供的MIDI乐谱库中选择一个乐谱并进行播放。	选择任意一个MIDI文件，点击play后即可在3D乐器上播放。	见下图4-7
用户在noteOnColor模块	设置好某个颜色后，在播	见下图4-8

下选择任意一个颜色和其相应的色彩亮暗度。	放MIDI文件时，按键按下后会显示为相应的颜色。	
用户在octave模块下通过鼠标点击或者键盘输入的方式改变八度音，有0-4 共5个音调进行选择。	设置好八度音阶即音调后，在键盘上再进行按键时，3D乐器上会显示出相应音阶按键的按下，且会播放出相应的音调音阶（具体声音不能在报告中展示，这里只展示相应音阶的按下）。	见下图4-9
用户在play与stop模块下通过鼠标点击相应按键，即可实现对选择乐谱文件的播放和暂停。	任一乐谱文件播放时，3D乐器会实时显示每一个音，并按下相应按键，且有声音播放出来；在暂停时，按键停止按下，声音也随即停止。	由于此块播放和暂停功能只用某个时刻的图很难表现出出来；而两个动作又都是瞬时发生实现的，故不再展示相关的结果图，说服力也不强。在这里还是期望用户可以主动尝试亲身体会。



图4-7　song模块选择任意一个MIDI文件进行播放测试结果

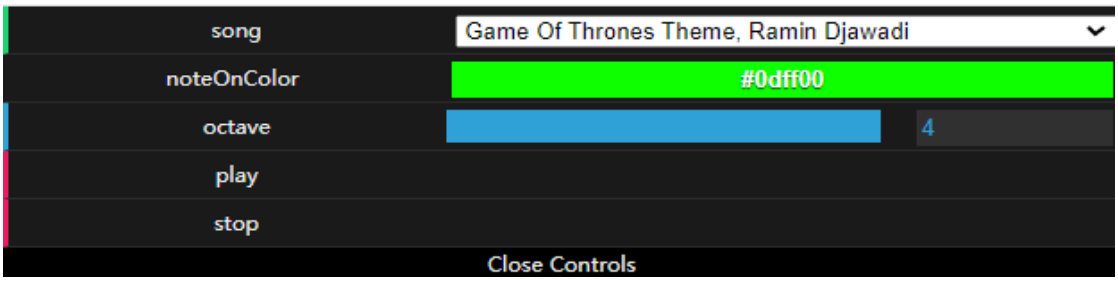


（a）



（b）

图4-8　noteOnColor模块下选择颜色，演奏时按键会显示相应颜色的测试结果



（a）



（b）

图4-9 octave模块下设置为最高的八度音，按键时3D乐器上相应展示测试结果

3. mp3到MIDI文件的转换实现

后端模块mp3到MIDI文件的转换实现的测试样例和预期测试结果如表4-3所示。

表4-3 mp3到midi文件的转换模块测试表

测试样例	预期结果	实际结果
用户上传的MP3文件： Zero.mp3	生成对应的MIDI文件 Zero.mid	生成对应的MIDI文件

将MP3文件的路径写入后端程序的路径变量中：

```
String inputFilePath = "D:\\桌面\\Zero.mp3";
```

运行后端程序如下图4-10所示：

```
size=    4360kB time=00:02:19.52 bitrate= 256.0kbits/s speed= 665x
video:0kB audio:4360kB subtitle:0kB other streams:0kB global headers:0kB muxing overhead: 0.000000%
Segment 0 / 25
Segment 1 / 25
Segment 2 / 25
Segment 3 / 25
Segment 4 / 25
Segment 5 / 25
Segment 6 / 25
Segment 7 / 25
Segment 8 / 25
Segment 9 / 25
Segment 10 / 25
Segment 11 / 25
Segment 12 / 25
Segment 13 / 25
Segment 14 / 25
Segment 15 / 25
Segment 16 / 25
Segment 17 / 25
Segment 18 / 25
Segment 19 / 25
Segment 20 / 25
Segment 21 / 25
Segment 22 / 25
Segment 23 / 25
Segment 24 / 25
Segment 25 / 25
OK
```

图 4-10　后端运行的实际结果图

在后端代码的相同目录下获得对应的 midi 文件如下图 4-11 所示：

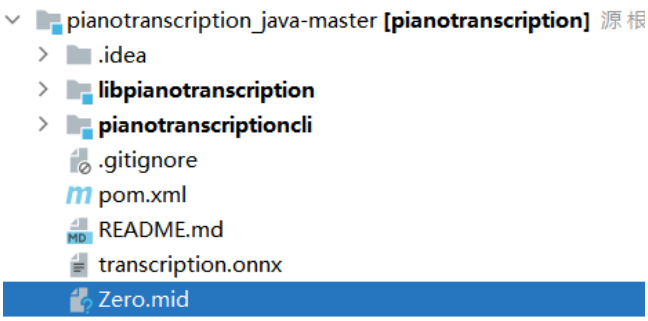


图 4-11 相同目录下获得对应的 midi 文件的实际结果图

4.4 结果分析

从上述各黑盒测试结果得出，web 应用实现了前端网页的搭建、乐器的 360°旋转、song 模块中提供的 MIDI 乐谱库的播放、noteOnColor 模块中按下琴键后的颜色的改变和亮暗度的调整、octave 模块中通过鼠标点击或者键盘输入的方式在 5 个选项中改变八度音和后端的 MP3 文件转 MIDI 文件。

因此，web 应用实现了用户上传 MP3 文件获得对应的 MIDI 文件并播放和用户自行弹奏钢琴两大功能。

5 总结

5.1 用户反馈

- 用户 1：Amazing! The songs in the MIDI library are all music that I am very familiar with. These classical piano songs are still difficult for me. But through this software, I can play and pause my favorite songs at any time. This makes it easier for me to see the key changes and gives me more opportunities to practice!（太奇妙了！MIDI 库里的歌曲都是我很熟悉的音乐，这些古典钢琴曲对于我来说还是有所难度的。不过通过这个软件，我可以随时对喜欢的歌曲播放和暂停。这使我更方便地看到琴键的变化，给我提供了更多的机会练习！）
- 用户 2：お気に入りのピアノ曲を視覚化できるとは想像もしていませんでした。音楽再生ソフトで素敵なピアノ曲を聴くと、どんな演奏になるか見たくなる。このソフトウェアは、この問題を非常にうまく解決しました。鍵盤が一つ一つ押されて美しい音色を奏でる様子は、まるで魔法のようです。（我从没想象过可以把我喜欢的钢琴歌曲可视化。当我在音乐播放软件上听到一首很好听的钢琴曲时，我是多么想看到它是如何弹出来的。这个软件很好地帮我解决了这个问题！看着一个一个琴键按下，演奏出美妙的音符，好像有一种魔力。）
- 用户 3：我本来以为这仅仅是一个更炫酷的播放器，却没想到钢琴是可以弹奏的，我既可以跟着钢琴的按键序列来学习钢琴曲，又可以自己在上面练习钢琴曲，而不必坐在一台钢琴前。在我看来，这个 web 应用是很具有创新性的，我从来没有见过和它可媲美的钢琴播放和弹奏一体化应用。

5.2 全文总结

下面是我们四人组根据开发过程中的经历对个人工作的总结(排名不分先后，按首字母排序）：

- **路昊东总结：**

在软件工程项目实现的过程中，小组四位成员讨论分配了任务，通过完成各自的任务，我们合作完成了这款应用。

作为小组的一名成员，我主要负责项目系统设计，系统完整实现等工作，同时负责说明文档编写，具体工作如下：

（1）完成了对于项目背景的部分调查，同时负责部分国内外竞品调研分析；

（2）完成系统结构的设计，编写代码实现服务端和客户端的基本框架；

（3）创建代码仓库，完成应用 README 等说明文档；

（4）与其他同学合作，分配好任务并具体实现一些模块：

- ❖ 服务端除音频文件转化外其他所有功能接口；
- ❖ 服务端和客户端通信部分；
- ❖ 云端资源分配；
- ❖ 客户端 MIDI 文件演奏和 3D 乐器模型导入逻辑。

（5）完成部分报告：撰写背景与意义、基本目标、系统结构设计、对象模型设计、数据管理说明、实现环境与代码管理、附录，以及全文总结和体会中属于自己的部分。

- **吴潇总结：**

经过小组四个人不懈的努力，我们最终完成了这个作品。

作为小组的一个成员，我从一开始就对这个题目十分感兴趣。我们从选题、查找资料，到归纳、策划等等过程，每一步我们都遇到了很多困难。在设计界面的时候，有些文字常常不能在我想要的位置上；界面的边角并没有放上准备的图标；页面标签上的图标也一度令我烦恼。但最终，我都一一克服了。

看到由我们写出来的软件，心里很激动。

具体工作如下：

（1）前端界面美化，基础场景设计。

（2）MP3 转 MIDI 操作的前后端连接。

（3）3D 模型搭建，构建合理的场景。

（4）向服务端发送客户请求。

- **薛江博总结：**

作为小组的一名成员，我主要负责的是后端模块的构建，以及系统测试等模块。在这大约一个多月的时间里，我们从组队到建立模型，再到逐步完成整个系统的搭建，再到接收反馈，不断调整完善界面、改正错误，这一步步走来十分的艰辛却又收获满满。

不过值得的是，最终我们完成了 3d-piano-player 的 Web 网页应用制作，实现了对于任意一段 mp3 文件，转换为 midi 文件并在 3d 钢琴上实时播放出来的功能，甚至还具有更深层次的实用价值，那就是在网页中就可以手把手教你弹奏

任何一段钢琴曲！

在收到朋友使用的反馈时，他们都连连称赞，认为我们的功能齐全而且方便快捷、实用意义大，而且这一方面的应用在国内的市场价值也很大。感谢每一个小组成员的认真努力，最后做出来的一个非常实用而且具有珍贵价值的东西真的很具有纪念意义。

具体工作如下：

(1) 实现后端 mp3 文件到 midi 文件的转换模块

(2) 完成 midi 文件存储记录模块

(3) 完成报告：设计完成 UML 相关需求分析图，包括系统用例图、用例交互图，进行了各个方面的可行性分析，并制定了开始的项目进度计划与最后的测试计划，完成测试了用例与后端关键函数的说明相关内容。

- **张子豪总结：**

身为小组队长，在本次项目开发中，我主要负责选题调研、NABCD 模型设计、代码测试、软件过程模型设计、UML 相关需求分析图设计和后端 MP3 文件转化为 MIDI 文件。具体工作如下：

（1）完成了对于项目基本目标以及项目可行性的分析，制定了立项阶段最终的项目方案。

（2）确定了软件过程模型为瀑布模型，按照可行性研究、需求分析、概要设计、详细设计、编码、调试、单元测试、集成测试、确定测试和运行维护的顺序来推进项目。

（3）设计了 UML 相关需求分析图，即 ER 图和数据流图。

（4）使用机器学习完成了后端 MP3 文件转化为 MIDI 文件的代码部分。

（5）完成了课程报告中需求分析概述、对象模型和关键数据结构的 MIDI Generator、关键算法设计的服务端模块、数据管理说明、测试计划和测试用例的 MP3 文件转 MIDI 文件、结果分析、总结的个人部分和体会的个人部分的撰写。

6 体会

本次软件工程课设对于我们四人小队的每一位成员都是一次奇妙而不可或缺的体验、疑惑旅程。自己设计并完全实现一个创新性的应用本就是一个特别酷炫特别有魅力的事情，我们在这一过程中也获得了独一无二的感悟与体会，并在这一部分逐一分享如下（排名不分先后，按首字母排序）：

6.1 路昊东

- **分工：**

  选题调研、系统设计和前后端框架搭建、前端 3D 模型处理、MIDI 文件演奏逻辑等功能实现、服务端功能实现、前后端交互功能实现

- **体会：**

  本次软件工程项目于我而言是一次享受创造、享受合作的极其愉悦的体验。

  首先不妨谈一下我们的 idea 的萌发、催生过程。

我们因对音乐共同的喜爱相聚，开始的选题暂定的听歌识谱，但是在项目开展、调研的过程中，我们研究了太多的播放器等同类竞品，但貌似都差强人意。随着了解的加深，我们加入了更多自己的想法，慢慢成熟后就形成了一个全新的产品，我们有自己的创意，也有自己的愿景，也相信这将是一个具有无限潜力的产品。这可能就是本次课设的意义之一吧。

再来谈谈具体的开发过程吧。

我的任务是负责系统的整体设计和整体框架的实现，具体框架完成后需要与队友合作完成具体模块的实现和优化。由于之前做过一些 web 端和移动端的开发项目，本人对相应的技术栈以及 Git 的使用也比较熟悉，所以在开发过程中还算得心应手，给大家分配商讨了各自的任务，也一同避免踩了很多坑。

然而，软件的实现过程虽然快乐也充满了艰辛，实现过程中暴露出了许许多多的问题，比如不切实际的技术栈、差强人意的官方代码库，甚至发现了初期设计的错误，不得不推倒重来。所幸，由于之前的经验，再次编写程序不再那么生疏，很快完成了软件的基本功能实现。完成任务后，我们发布了网站，并从国内外一些朋友的使用中得到了一些反馈，也获得到了老师的建议。讨论过后，我们继续完善了原先的应用，使得它的功能更加完善，特色更加明显，界面也更加讨喜，也提高了代码的鲁棒性。

当然本次开发也让我搜集并学习了很多新知识，应用了许多新的技术栈。前期调研中我们本来打算使用 C++处理 MIDI 文件，所以研究了大量的 C++库和开

源代码库，最终为了整体实现的简洁和高效使用了 Java，并在处理文件的过程中学习并使用了不少机器学习的知识点。在前端的实现过程中，我们找到了 three.js 和 MIDI.js 这两个功能强大的开源代码库，看了很多英文文档，也慢慢摸索出来了其使用方法，实现了自己所需的功能。

最后也是最重要的，是致谢。我十分感谢我的 3 位队友。我们一起在这个项目中一步步前行，一个大软件工程项目如果要做好，单靠个人的能力是不可能完成的。并且团队内的配合十分重要，我为认识了这样默契十足的队友感到开心；同时也感谢老师和助教给我们的帮助！

6.2 吴潇

- **分工：**

  题材调研、前端界面设计与美化，原型图设计。

- **体会：**

本次软件工程的项目给我提供了很好的学习前端的机会。从基础的 html 语言，每一部分是怎么写的。我通过在网上的一些教学网站，逐渐熟练地运用了各种各样的语法。以前使用过那么多的网页，没想到我也可以写出来。

我还学会了在任何页面按下 F12，就可以进入开发者选项，以至于可以看到开发人员编写的前端代码。这可以让我更加深入地了解到某些语句的作用，以及它们之间的联系。

通过使用 CSS 等样式表，可以在 html 文档里添加更多有趣的东西。我可以更方便地完善整个界面的内容。我学到了将它放在 head 部分，就可以调用其存储的内容。

我很高兴接触到了 Three.js——一个强大的 3D 模型库。里面有各种各样的基础几何，还有光源、影子等有趣的东西。我很幸运地为虚拟钢琴添加了一些我喜欢的元素：关于每个琴键按下的角度、影子的方向等。我也很高兴客户喜欢它们。

我们小组刚开始并没有确定下来选题，不过大家对于听曲识谱都比较感兴趣。虽然这个过程中出现了很多困难，但是大部分都被我们一一克服了。发现问题并解决问题的感觉很奇妙，让我感受到是在创造一些前所未有的东西。

我是很喜欢音乐的，通过这个项目，我从另一个角度更加深刻地了解到了它。MP3 文件和 MIDI 文件之间地奥秘，几行二进制数字便可以把乐曲的所有信息都保存下来，真的是很奇妙。我之前尝试过把 MIDI 文件转成乐谱，虽然这需要一

些制谱软件的帮助，不过效果还是不错的。

有几位用户也尝试过了我们的成果，这令我十分荣幸。整个界面对于我来说就像是一块画布，我用一行一行代码为其增添色彩。我学会了 h1-h5 不同级别的标题，以及整个 html 中各个部分的作用。我曾一度因为中文汉字编码问题而头疼，查阅资料后发现需要在代码开头添加 charset = 'UTF-8'。通过这次实践，我也学会了如何在一个项目里和队员一起分工合作，学会了用 git 命令向 github 仓库提交命令、更新命令。虽然刚开始还不算太熟练，但慢慢地，我做的越来越快。

在写报告地过程中，我又熟悉了对类图的运用。我还学会了在墨刀上画原型图，这真的是一个很方便的软件，很直观地把我们想要的东西都做了出来。

6.3 薛江博

- **分工：**

  后端 mp3 文件到 midi 文件的转换模块、midi 文件存储记录模块、UML 相关需求分析图、进度计划与系统测试

- **体会：**

本次软件工程项目是我第一次实际参与到的软件开发工程，在各个阶段都遇到了不少的困难，但最终也得到了许多学习收获。由于是初次参与软件开发，对于各个系统流程和具体实现的过程都不了解，通过上课倾听老师的讲解介绍、以及课下组内成员之间时常的沟通交流，后来一步步明确了工作步骤，初步制定了项目进度计划，明确了这次要制作的软件的主要系统结构。并且通过交流分析，每个人根据自己较擅长的领域实现各自的分工安排，最后决定由路昊东和吴潇同学负责前端界面的设计模块，我与张子豪同学实现后端 mp3 文件到 midi 文件的转换模块、midi 文件存储记录模块。

在确定各自的分工模块后，我们便开始了紧锣密鼓的工作。首先为了完成这一个相关音乐软件应用的制作，我们需要对相关的音乐知识有一定的了解和认识，虽然自己有一些钢琴吉他基础，但是对于这次要实现的工作，对于 mp3 文件、midi 文件在计算机中以何种形式何种编码进行存储和记录却是没有什么了解。后来通过查找有关资料，我逐渐了解到 mp3 的全称是 MPEG Audio Layer3,它是一种高效的计算机音频编码方案,它以较大的压缩比将音频文件转换成较小的扩展名为.mp3 的文件,基本保持原文件的音质。mp3 音频压缩包含编码和解码两个部分。mp3 采用了感知音频编码这一失真算法，它截掉了大量的冗余信号和无关

的信号,编码器通过混合滤波器组将原始声音变换到频率域,再经过量化,转换成 Huffman 编码,形成 mp3 位流。解码器要简单得多,它的任务是从编码后的谱线成分中,经过反量化和逆变换,提取出声音信号。最后通过对 mp3 文件结构的解析和提取，为下一步的转换 midi 文件做好了准备工作。相较于 mp3 文件，我对于 mid 文件的了解就更少了，midi 是编曲界最广泛的音乐标准格式，可称为“计算机能理解的乐谱”。结合 midi 数据流、编码和常用 midi 消息等内容，在后端的具体实现中，利用到了 MidiSystem 类提供对已安装的 midi 系统资源的访问，包括合成器、音序器和 midi 输入和输出端口等设备。该类还具有用于读取包含标准 midi 文件数据或音库的文件、流和 URL 的方法。我们可以在 MidiSystem 中查询指定 midi 文件的格式。并在后端的具体实现中为 MidiMessage 设置了四个参数：time（时间记录）、 midinote（midi 记录） or control change(64)（共 64 个音色库可进行选择）、 velocity（速度） or value（与 time 共同构成一个以刻度表示的相应时间戳）、 type （0 表示 midi note ; 1 表示 control change）。

后续通过一点点的实现和修改问题，经过反复不断的调试，最后实现了对于 mp3 文件和 midi 文件的解析和存储记录。后面就是 mp3 to midi 的功能实现模块，最开始时对于这一块的实现并不能找到有关的开源代码学习借鉴，以至于我们对于这种知识盲区的探索设计一度陷入一筹莫展的境地，其实本来我们选题的是听音识谱，但碍于目前的技术水平和可供查阅资料的限制，我们选择了一种更为灵活的听音识谱方式，也就是在 3d 钢琴上将转换出的 midi 乐谱实时的播放演奏出来。后面在前端两位同学费尽心思、尽心竭力的初步制作完成，后续要继续添加更多的模块功能，我们两位后端同学也加紧了步伐，由于转换功能模块的实现是一个很大的项目工程，所以在我完成 mp3、midi 文件的解析记录等工作后，也加入到了张子豪同学的工作中，从他那里得知可以使用到机器学习方面的有关算法，对于机器学习相关知识没有充分了解认识的我开始时也遇到了一些困难，后来通过查阅资料并向同学交流取经后，两人协作最终把 mp3 到 midi 转换的工作系统的实现了出来，当然后续也进一步结合前端的构造和使用进行了进一步的优化和调整。

对我来说还有一个比较大的挑战就是对于 git 分布式管理系统的使用不熟悉。本次开发是我第一次参与基于 git 的多人协同开发，这里要感谢组内成员路昊东同学对此的熟练掌握并耐心细致地给了我很多指导，让我最后得以成功地使用了 git 实现个人工作的提交。虽然学习使用 git 花费了一定的时间，但 git 的使用极大的方便了我们对开发的管理，方便我们实现代码的合并和版本控制。另外，模块化的编程也为我们的开发提供了巨大的便利。我们不需要关心其他人负责的

模块的实现方式，而只需相关内容进行调用即可，让我感受到模块化编程的重要作用。

6.4 张子豪

- **分工：**

  选题调研、NABCD 模型设计、代码测试、软件过程模型设计、UML 相关需求分析图设计和后端 MP3 文件转化为 MIDI 文件。

- **体会：**

在本次软件工程课程以及任务的完成过程中，我第一次参与了软件的设计，
实现，测试等流程，最终通过合作完成了一个完整的 web 项目。在这一过程中，我遇到了许多困难，也有了许多收获。

我们团队很快就完成了选题，但由于我们对于音乐并不太了解，因此项目进度停滞了一段时间，四人一度打算换一个选题，但最终我们快速了解了各个方面的知识：Java、Javascript、css、html,经过审慎研究，我们坚持了这个选题。

对于软件过程模型的选择是定下选题的第一件事情。一共有 3 种模型可供选择：瀑布模型、通用过程模型和专用模型。经过一番研究，我们决定按照瀑布模型来开发 web 应用，也就是按照可行性研究、需求分析、概要设计、详细设计、编码、调试、单元测试、集成测试、确定测试和运行维护的顺序来推进项目。其中单元测试包括两部分，后端 MP3 转 MIDI 文件的单元测试和前端上传 MP3 文件、下载 MIDI 文件和沟通后端的功能的单元测试。

由于没有音乐基础，我最初对于 MP3 文件和 MIDI 文件并没有了解，因而不清楚其特征和编码方式。经过长期的学习，我了解到 MP3 既是一种音频封装格式，又是一种音频编码格式，ffmpeg 提供了 MP3 音频解码器，将其解码为 PCM 格式，在 java 中可以使用数组来保存。而 PCM 转换为 MIDI 文件,经过搜索资料，没找到传统的转换算法，但我发现可以使用机器学习算法，使用机器学习获得 onnx 文件来存储训练好的模型，在 java 程序中直接调用。这样就完成了 MP3 文件转化为 MIDI 文件。

对于 UML 相关需求分析图设计，我最初并不了解。经过上课和研读课本，我知道了 UML 通过图形化的表示机制从多个侧面对系统的分析和设计模型进行刻画，它定义了 5 类图形机制：用例视图、结构视图、行为视图、构件视图和部署视图。我经过一番研究，画出了 ER 图和数据流图。

成功实现 MP3 文件转化为 MIDI 文件让我很激动，因为这是我第一次完成这样的大型的需求。但随之而来的是转化后的 MIDI 文件的低准确率，我一次又一次修改机器学习代码的参数和训练模型，期间还研究了谷歌的 MT3:Multi-Task Multitrack Music Transcription 项目的参数设置，最终将参数修改到了合适的值，成功提高了 MIDI 文件的准确率。这个过程耗了我最多时间，调参数确实是很枯燥无味但又很有价值的。

附录 参考文献

[1] Curtis Hawthorne, Ian Simon, Rigel Swavely, Ethan Manilow, and Jesse Engel. “Sequence-to-sequence Piano Transcription With Transformers”. In: International Society for Music Information Retrieval (2021).

[2] Josh Gardner* , Ian Simon, Ethan Manilow† , Curtis Hawthorne, Jesse Engel. “MT3: Mutlti-Task Multitrack Music Transcription”. In: International Conference on Learning Representation (2022).

[3] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, D. P. Ellis, and C. C. Raffel, “mir_eval: A transparent implementation of common MIR metrics”. In International Society for Music Information Retrieval (2014).

[4] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: https://www.tensorflow.org/ .

[5] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin et al., “Tensorflow: Large-scale machine learning on heterogeneous distributed systems,” arXiv preprint arXiv:1603.04467 (2016) .

[6] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, “This time with feeling: Learning expressive musical performance,” Neural Computing and Applications, vol. 32, no. 4, pp. 955–967, 2020.

[7] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in International Conference on Learning Representations, 2021. [Online]. Available: https://openreview.net/forum?id = YicbFdNTTy.